

The survival Package

April 28, 2002

Title Survival analysis, including penalised likelihood.

Maintainer Thomas Lumley <tlumley@u.washington.edu>

Priority recommended

Version 2.9-1

Requires R ($\geq 1.4.0$)

Author S original by Terry Therneau, ported by Thomas Lumley

Description survival analysis: descriptive statistics, two-sample tests, parametric accelerated failure models, Cox model. Delayed entry (truncation) allowed for all models; interval censoring for parametric models.

License GPL2

R topics documented:

Surv	2
aml	4
as.date	4
attrassign	5
bladder	6
clogit	7
cluster	8
colon	8
cox.zph	9
coxph	10
coxph.detail	13
coxph.object	14
date.ddmmyy	15
date.mdy	16
date.mmddyy	16
date.mmddyyyy	17
date.object	18
frailty	18
heart	20
is.ratetable	21
kidney	21
lines.survfit	22

lung	24
mdy.date	24
ovarian	25
pbc	26
plot.cox.zph	27
plot.survfit	28
predict.coxph	30
predict.survreg	31
print.survfit	32
pspline	34
pyears	35
ratetable	37
rats	38
residuals.coxph	38
residuals.survreg	40
ridge	41
stanford2	42
strata	42
summary.survfit	43
survdiff	44
survexp	46
survexp.fit	48
ratetables	49
survfit	50
survfit.object	53
survival-internal	54
survobrien	54
survreg	55
survreg.control	57
survreg.distributions	58
survreg.object	59
survreg.old	60
tcut	61
untangle.specials	62
veteran	63

Index	64
--------------	-----------

Surv	<i>Create a Survival Object</i>
------	---------------------------------

Description

Create a survival object, usually used as a response variable in a model formula.

Usage

```
Surv(time, event)
or
Surv(time, time2, event, type=<<see below>>, origin=0)
is.Surv(x)
```

Arguments

time	for right censored data, this is the follow up time. For interval data, the first argument is the starting time for the interval.
x	any S-PLUS object.
event	The status indicator, normally 0=alive, 1=dead. Other choices are T/F (TRUE = death) or 1/2 (2=death). For interval censored data, the status indicator is 0=right censored, 1= event at time , 2=left censored, 3=interval censored. Although unusual, the event indicator can be omitted, in which case all subjects are assumed to have an event.
time2	ending time of the interval for interval censored or counting process data only. Intervals are assumed to be open on the left and closed on the right, (start , end]. For counting process data, event indicates whether an event occurred at the end of the interval.
type	character string specifying the type of censoring. Possible values are "right", "left", "counting", "interval", or "interval2". The default is "right" or "counting" depending on whether the time2 argument is absent or present, respectively.
origin	for counting process data, the hazard function origin. This is most often used in conjunction with a model containing time dependent strata in order to align the subjects properly when they cross over from one strata to another.

Value

An object of class **Surv**. There are methods for **print**, **is.na**, and subscripting survival objects. To include a survival object inside a data frame, use the **I()** function. **Surv** objects are implemented as a matrix of 2 or 3 columns.

In the case of **is.Surv**, a logical value T if **x** inherits from class "Surv", otherwise an F.

DETAILS

In theory it is possible to represent interval censored data without a third column containing the explicit status. Exact, right censored, left censored and interval censored observation would be represented as intervals of (a,a), (a, infinity), (-infinity,b), and (a,b) respectively; each specifying the interval within which the event is known to have occurred.

If **type** = "interval2" then the representation given above is assumed, with NA taking the place of infinity. If 'type="interval" **event** must be given. If **event** is 0, 1, or 2, the relevant information is assumed to be contained in **time**, the value in **time2** is ignored, and the second column of the result will contain a placeholder.

Presently, the only methods allowing interval censored data are the parametric models computed by **survreg**, so the distinction between open and closed intervals is unimportant. The distinction is important for counting process data and the Cox model.

The function tries to distinguish between the use of 0/1 and 1/2 coding for left and right censored data using **if (max(status)==2)**. If 1/2 coding is used and all the subjects are censored, it will guess wrong. Use 0/1 coding in this case.

See Also

[coxph](#), [survfit](#), [survreg](#).

Examples

```
data(aml)
Surv(aml$time, aml$status)
```

<code>aml</code>	<i>Acute Myelogenous Leukemia survival data</i>
------------------	---

Description

Survival in patients with Acute Myelogenous Leukemia

Usage

```
data(aml)
data(leukemia)
```

Format

```
time:  survival or censoring time
status: censoring status
x:      maintenance chemotherapy given? (factor)
```

Source

Miller "Survival Analysis"

<code>as.date</code>	<i>Coerce Data to Dates</i>
----------------------	-----------------------------

Description

Converts any of the following character forms to a Julian date: 8/31/56, 8-31-1956, 31 8 56, 083156, 31Aug56, or August 31 1956.

Usage

```
as.date(x, order = "mdy", ...)
```

Arguments

```
x          input data vector.
order      if x is character, defines the order in which the terms are assumed to
           appear in a xx/xx/xx date. The default is month/day/year; any permutation
           of mdy is legal.
...        if x is character, then any other arguments from mdy.date() can be used
           as well.
```

Details

If `x` is numeric, then `floor(x)` is returned, e.g., `as.date(35)` is the same as `as.date(35.2)` and gives Feb 2, 1960. If `x` is character, the program attempts to parse it.

Value

For each date, the number of days between it and January 1, 1960. The date will be missing if the string is not interpretable.

See Also

[mdy.date](#), [date.mmddyy](#), [date.ddmmyy](#)

Examples

```
as.date(c("1jan1960", "2jan1960", "31mar1960", "30jul1960"))
```

attrassign	<i>Create new-style "assign" attribute</i>
------------	--

Description

The "assign" attribute on model matrices describes which columns come from which terms in the model formula. It has two version. R uses the original version, but the newer version is sometimes useful.

Usage

```
attrassign(object, tt,...)
attrassign(object,...)
```

Arguments

<code>object</code>	model matrix or linear model object
<code>tt</code>	terms object
<code>...</code>	ignored

Value

A list with names corresponding to the term names and elements that are vectors indicating which columns come from which terms

See Also

[terms](#), [model.matrix](#)

Examples

```
data(pbc)
formula<-Surv(time,status)~factor(edtrt)
tt<-terms(formula)
mf<-model.frame(tt,data=pbc)
mm<-model.matrix(tt,mf)
## a few rows of data
mm[1:3,]
## old-style assign attribute
attr(mm,"assign")
## new-style assign attribute
attrassign(mm,tt)
```

bladder

Bladder Cancer Recurrences

Description

Data on recurrences of bladder cancer, used by many people to demonstrate methodology for recurrent event modelling.

Usage

```
data(bladder)
```

Format

bladder

id:	Patient id
rx:	Treatment 1=placebo 2=thiotepa
number:	Initial number of tumours (8=8 or more)
size:	size (cm) of largest initial tumour
stop:	recurrence or censoring time
enum:	which recurrence (up to 4)

bladder2

id:	Patient id
rx:	Treatment 1=placebo 2=thiotepa
number:	Initial number of tumours (8=8 or more)
size:	size (cm) of largest initial tumour
start:	start of interval (0 or previous recurrence time)
stop:	recurrence or censoring time
enum:	which recurrence (up to 4)

Source

Wei, Lin, Weisfeld, JASA 1989

clogit*Conditional logistic regression*

Description

Estimates a logistic regression model by maximising the conditional likelihood. Uses a model formula of the form `case.status~exposure+strata(matched.set)`. The default is to use the exact conditional likelihood, a commonly used approximate conditional likelihood is provided for compatibility with older software.

Usage

```
clogit(formula, data, method=c("exact", "approximate"), na.action=getOption("na.action"), subset)
```

Arguments

<code>formula</code>	Model formula
<code>data</code>	data frame
<code>method</code>	correct or approximate conditonal likelihood?
<code>na.action</code>	missing value handling
<code>subset</code>	subset of records to use

Value

An object of class `"clogit"`, which is a wrapper for a `"coxph"` object.

Author(s)

Thomas Lumley

See Also

[strata](#), [coxph](#), [glm](#)

Examples

```
data(infert)
clogit(case~spontaneous+induced+strata(stratum),data=infert)
```

cluster

Identify Clusters

Description

This is a special function used in the context of the Cox model. It identifies correlated groups of observations, and is used on the right hand side of a formula.

Usage

```
cluster(x)
```

Arguments

x A character, factor, or numeric variable.

Details

The function's only action is semantic, to mark a variable as the cluster indicator.

Value

x

See Also

[coxph](#), [Surv](#)

Examples

```
coxph(Surv(futime, fustat) ~ age + cluster(group), data = ovarian)
```

colon

Chemotherapy for Stage B/C colon cancer

Description

These are data from one of the first successful trials of adjuvant chemotherapy for colon cancer. Levamisole is a low-toxicity compound previously used to treat worm infestations in animals; 5-FU is a moderately toxic (as these things go) chemotherapy agent. There are two records per person, one for recurrence and one for death

Format

id: id
 study: 1 for all patients
 rx: Treatment - Obs(ervation), Lev(amisole), Lev(amisole)+5-FU
 sex: 1=male
 age: in years
 obstruct: obstruction of colon by tumour
 perfor: perforation of colon
 adhere: adherence to nearby organs
 nodes: number of lymph nodes with detectable cancer
 status: censoring status
 differ: differentiation of tumour (1=well, 2=moderate, 3=poor)
 extent: Extent of local spread (1=submucosa, 2=muscle, 3=serosa, 4=contiguous structures)
 surg: time from surgery to registration (0=short, 1=long)
 node4: more than 4 positive lymph nodes
 time: days until death
 etype: 1=recurrence,2=death

Source

Danyu Lin

cox.zph

Test the Proportional Hazards Assumption of a Cox Regression

Description

Test the proportional hazards assumption for a Cox regression model fit (`coxph`).

Usage

```
cox.zph(fit, transform="km", global=T)
```

Arguments

fit	the result of fitting a Cox regression model, using the <code>coxph</code> function.
transform	a character string specifying how the survival times should be transformed before the test is performed. Possible values are "km", "rank", "identity" or a function of one argument.
global	should a global chi-square test be done, in addition to the per-variable tests.
x	if true, then the result will be a list containing the test table (a matrix), x and y. If false then only the test table is returned.
table	a matrix with one row for each variable, and optionally a last row for the global test. Columns of the matrix contain the correlation coefficient between transformed survival time and the scaled Schoenfeld residuals, a chi-square, and the two-sided p-value. For the global test there is no appropriate correlation, so an NA is entered into the matrix as a placeholder.

<code>x</code>	the transformed time axis.
<code>y</code>	the matrix of scaled Schoenfeld residuals. There will be one column per variable and one row per event. The row labels contain the original event times (for the identity transform, these will be the same as <code>x</code>).
<code>call</code>	the calling sequence for the routine. The computations require the original <code>x</code> matrix of the Cox model fit. Thus it saves time if the <code>x=T</code> option is used in <code>coxph</code> . This function would usually be followed by both a plot and a print of the result. The plot gives an estimate of the time-dependent coefficient <code>beta(t)</code> . If the proportional hazards assumption is true, <code>beta(t)</code> will be a horizontal line. The printout gives a test for <code>slope=0</code> .

Value

an object of class "`cox.zph`", with components:

Note

The `plot` method requires the `splines` library

References

P. Grambsch and T. Therneau (1994), Proportional hazards tests and diagnostics based on weighted residuals. *Biometrika*, **81**, 515-26.

See Also

[coxph](#), [Surv](#).

Examples

```
data(ovarian)
fit <- coxph( Surv(futime, fustat) ~ age + rx, ovarian)
temp<- cox.zph(fit)
print(temp)           #display the results
plot(temp)            #plot curves
```

`coxph`

Fit Proportional Hazards Regression Model

Description

Fits a Cox proportional hazards regression model. Time dependent variables, time dependent strata, multiple events per subject, and other extensions are incorporated using the counting process formulation of Andersen and Gill.

Usage

```
coxph(formula, data=parent.frame(), weights, subset,
      na.action, init, control, method=c("efron","breslow","exact"),
      singular.ok=TRUE, robust=FALSE,
      model=FALSE, x=FALSE, y=TRUE,...
      )
```

Arguments

<code>formula</code>	a formula object, with the response on the left of a <code>~</code> operator, and the terms on the right. The response must be a survival object as returned by the <code>Surv</code> function.
<code>data</code>	a data.frame in which to interpret the variables named in the <code>formula</code> , or in the <code>subset</code> and the <code>weights</code> argument.
<code>subset</code>	expression saying that only a subset of the rows of the data should be used in the fit.
<code>na.action</code>	a missing-data filter function, applied to the model.frame, after any subset argument has been used. Default is <code>options()\$na.action</code> .
<code>weights</code>	case weights.
<code>eps</code>	convergence threshold. Iteration will continue until the relative change in the log-likelihood is less than <code>eps</code> . Default is <code>.0001</code> .
<code>init</code>	vector of initial values of the iteration. Default initial value is zero for all variables.
<code>control</code>	Object of class <code>coxph.control</code> specifying iteration limit and other control options. Default is <code>coxph.control(...)</code> .
<code>method</code>	a character string specifying the method for tie handling. If there are no tied death times all the methods are equivalent. Nearly all Cox regression programs use the Breslow method by default, but not this one. The Efron approximation is used as the default here, as it is much more accurate when dealing with tied death times, and is as efficient computationally. The exact method computes the exact partial likelihood, which is equivalent to a conditional logistic model. If there are a large number of ties the computational time will be excessive.
<code>singular.ok</code>	logical value indicating how to handle collinearity in the model matrix. If <code>TRUE</code> , the program will automatically skip over columns of the X matrix that are linear combinations of earlier columns. In this case the coefficients for such columns will be NA, and the variance matrix will contain zeros. For ancillary calculations, such as the linear predictor, the missing coefficients are treated as zeros.
<code>robust</code>	if <code>TRUE</code> a robust variance estimate is returned. Default is <code>TRUE</code> if the model includes a <code>cluster()</code> operative, <code>FALSE</code> otherwise.
<code>model</code>	flags to control what is returned. If these are true, then the model frame, the model matrix, and/or the response is returned as components of the fitted model, with the same names as the flag arguments.
<code>x</code>	Return the design matrix in the model object?
<code>y</code>	return the response in the model object?
<code>...</code>	Who knows?

Details

The proportional hazards model is usually expressed in terms of a single survival time value for each person, with possible censoring. Andersen and Gill reformulated the same problem as a counting process; as time marches onward we observe the events for a subject, rather like watching a Geiger counter. The data for a subject is presented as multiple rows or "observations", each of which applies to an interval of observation (start, stop].

Value

an object of class "coxph". See `coxph.object` for details.

Side Effects

Depending on the call, the `predict`, `residuals`, and `survfit` routines may need to reconstruct the x matrix created by `coxph`. Differences in the environment, such as which data frames are attached or the value of `options()$contrasts`, may cause this computation to fail or worse, to be incorrect. See the survival overview document for details.

SPECIAL TERMS

There are two special terms that may be used in the model equation. A 'strata' term identifies a stratified Cox model; separate baseline hazard functions are fit for each strata. The `cluster` term is used to compute a robust variance for the model. The term `+ cluster(id)`, where `id == unique(id)`, is equivalent to specifying the `robust=T` argument, and produces an approximate jackknife estimate of the variance. If the `id` variable were not unique, but instead identifies clusters of correlated observations, then the variance estimate is based on a grouped jackknife.

CONVERGENCE

In certain data cases the actual MLE estimate of a coefficient is infinity, e.g., a dichotomous variable where one of the groups has no events. When this happens the associated coefficient grows at a steady pace and a race condition will exist in the fitting routine: either the log likelihood converges, the information matrix becomes effectively singular, an argument to `exp` becomes too large for the computer hardware, or the maximum number of interactions is exceeded. The routine attempts to detect when this has happened, not always successfully.

PENALISED REGRESSION

`coxph` can now maximise a penalised partial likelihood with arbitrary user-defined penalty. Supplied penalty functions include ridge regression ([ridge](#)), smoothing splines ([pspline](#)), and frailty models ([frailty](#)).

References

P. Andersen and R. Gill. "Cox's regression model for counting processes, a large sample study", *Annals of Statistics*, 10:1100-1120, 1982.

T. Therneau, P. Grambsch, and T. Fleming. "Martingale based residuals for survival models", *Biometrika*, March 1990.

See Also

[cluster](#), [survfit](#), [Surv](#), [strata](#), [ridge](#), [pspline](#), [frailty](#).

Examples

```
# Create the simplest test data set
#
test1 <- list(time= c(4, 3,1,1,2,2,3),
               status=c(1,NA,1,0,1,1,0),
               x=    c(0, 2,1,1,1,0,0),
               sex=   c(0, 0,0,0,1,1,1))
```

```

coxph( Surv(time, status) ~ x + strata(sex), test1) #stratified model

#
# Create a simple data set for a time-dependent model
#
test2 <- list(start=c(1, 2, 5, 2, 1, 7, 3, 4, 8, 8),
              stop =c(2, 3, 6, 7, 8, 9, 9, 9,14,17),
              event=c(1, 1, 1, 1, 1, 1, 1, 0, 0, 0),
              x     =c(1, 0, 0, 1, 0, 1, 1, 1, 0, 0) )

summary( coxph( Surv(start, stop, event) ~ x, test2))

```

coxph.detail

Details of a Cox Model Fit

Description

Details of a Cox model fit. Returns the individual contributions to the first and second derivative matrix, at each unique event time.

Usage

```
coxph.detail(object)
```

Arguments

object a Cox model object, i.e., the result of `coxph`.

Details

This function may be useful for those who wish to investigate new methods or extensions to the Cox model. The example below shows one way to calculate the Schoenfeld residuals.

Value

a list with components

time	the vector of unique event times
nevent	the number of events at each of these time points.
means	a matrix with one row for each event time and one column for each variable in the Cox model, containing the weighted mean of the variable at that time, over all subjects still at risk at that time. The weights are the risk weights <code>exp(x %*% fit\$coef)</code> .
nrisk	number of subjects at risk.
hazard	the hazard increment.
score	the contribution to the score vector (first derivative of the log partial likelihood) at each time point.
imat	the contribution to the information matrix (second derivative of the log partial likelihood) at each time point.
varhaz	the variance of the hazard increment.

x,y	copies of the input data.
strata	only present for a stratified Cox model, this is a table giving the number of time points of component time that were contributed by each of the strata.

See Also

[coxph](#), [residuals.coxph](#)

Examples

```
data(ovarian)
fit <- coxph(Surv(futime,fustat) ~ age + rx + ecog.ps, ovarian, x=T)
fitd <- coxph.detail(fit)
events <- fit$y[,2]==1
etime <- fit$y[events,1] #the event times --- may have duplicates
indx <- match(etime, fitd$time)
sresid <- fit$x[events,] - fitd$means[indx,]
```

coxph.object	<i>Proportional Hazards Regression Object</i>
--------------	---

Description

This class of objects is returned by the **coxph** class of functions to represent a fitted proportional hazards model.

Objects of this class have methods for the functions **print**, **summary**, **residuals**, **predict** and **survfit**.

COMPONENTS

The following components must be included in a legitimate **coxph** object.

coefficients the coefficients of the linear predictor, which multiply the columns of the model matrix. If the model is over-determined there will be missing values in the vector corresponding to the redundant columns in the model matrix.

var the variance matrix of the coefficients. Rows and columns corresponding to any missing coefficients are set to zero.

naive.var this component will be present only if the **robust** option was true. If so, the **var** component will contain the robust estimate of variance, and this component will contain the ordinary estimate.

loglik a vector of length 2 containing the log-likelihood with the initial values and with the final values of the coefficients.

score value of the efficient score test, at the initial value of the coefficients.

rscore the robust log-rank statistic, if a robust variance was requested.

wald.test the Wald test of whether the final coefficients differ from the initial values.

iter number of iterations used.

linear.predictors the vector of linear predictors, one per subject.

residuals the martingale residuals.

means vector of column means of the X matrix. Subsequent survival curves are adjusted to this value.

n the number of observations used in the fit.

weights the vector of case weights, if one was used.

method the computation method used.

na.action the `na.action` attribute, if any, that was returned by the `na.action` routine.

The object will also contain the following, for documentation see the `lm` object: **terms**, **assign**, **formula**, **call**, and, optionally, **x**, **y**, and/or **frame**.

See Also

[coxph](#), [coxph.detail](#), [cox.zph](#), [survfit](#), [residuals.coxph](#), [survreg](#)

<code>date.ddmmyy</code>	<i>Format a Julian date</i>
--------------------------	-----------------------------

Description

Given a vector of Julian dates, this returns them in the form “10Nov89”, “28Jul54”, etc.

Usage

```
date.ddmmyy(sdate)
```

Arguments

sdate A vector of Julian dates, e.g., as returned by `mdy.date()`.

Value

A vector of character strings containing the formatted dates.

See Also

[mdy.date](#), [date.mdy](#)

Examples

```
date.ddmmyy(1:10)
```

date.mdy

Convert from Julian Dates to Month, Day, and Year

Description

Convert a vector of Julian dates to a list of vectors with the corresponding values of month, day and year, and optionally weekday.

Usage

```
date.mdy(sdate, weekday = FALSE)
```

Arguments

sdate	a Julian date value, as returned by <code>mdy.date()</code> , number of days since 1/1/1960.
weekday	if TRUE, then the returned list also will contain the day of the week (Sunday=1, Saturday=7).

Value

A list with components `month`, `day`, and `year`.

References

Press, W. H., Teukolsky, S. A., Vetterling, W. T., and Flannery, B. P. (1992). *Numerical Recipes: The Art of Scientific Computing (Second Edition)*. Cambridge University Press.

Examples

```
day <- 7
temp <- date.mdy(mdy.date(month = 7, day = day, year = 1960))
## Check for illegal dates, such as 29 Feb in a non leap year
if (temp$day != day) {
  cat("Some illegal dates\n")
} else {
  cat("All days are legal\n")
}
```

date.mmdyy

Format a Julian date

Description

Given a vector of Julian dates, this returns them in the form “10/11/89”, “28/7/54”, etc.

Usage

```
date.mmdyy(sdate, sep = "/")
```


Arguments

- sdate** A vector of Julian dates, e.g., as returned by `mdy.date()`.
- sep** Character string used to separate the month, day, and year portions of the returned string.

Value

A vector of character strings containing the formatted dates.

See Also

[date.mdy](#), [mdy.date](#), [date.ddmmyy](#)

Examples

```
date.mmddyy(as.date(10))
```

date.mmddyyyy	<i>Format a Julian date</i>
---------------	-----------------------------

Description

Given a vector of Julian dates, this returns them in the form “10/11/1989”, “28/7/1854”, etc.

Usage

```
date.mmddyyyy(sdate, sep = "/")
```

Arguments

- sdate** A vector of Julian dates, e.g., as returned by `mdy.date()`.
- sep** Character string used to separate the month, day, and year portions of the returned string.

Value

A vector of character strings containing the formatted dates.

See Also

[date.mdy](#), [mdy.date](#), [date.ddmmyy](#)

Examples

```
date.mmddyyyy(as.date(1:10))
```

<code>date.object</code>	<i>Date Objects</i>
--------------------------	---------------------

Description

Objects of class `"date"`.

Usage

```
is.date(x)
```

Arguments

`x` any R object.

Details

Dates are stored as the number of days since 1/1/1960, and are kept in integer format. (This is the same baseline value as is used by SAS). The numerical methods for dates treat `date - date` as a numeric, and `date +- numeric` as a date.

`is.date` returns `TRUE` if `x` has class `"date"`, and `FALSE` otherwise. Its behavior is unaffected by any attributes of `x`; for example, `x` could be a date array (in contrast to the behavior of `is.vector`).

`as.date` returns `x` if `x` is a simple object of class `"date"`, and otherwise a date vector of the same length as `x` and with data resulting from coercing the elements of `x` to class `"date"`. See the manual page for `as.date()` for details.

Logical operations as well as the numeric functions `exp()`, `log()`, and so on are invalid.

Other methods exist for missing value, `as.character()`, printing, and summarizing.

See Also

[date.mdy](#), [mdy.date](#), [date.ddmmyy](#), [as.date](#).

<code>frailty</code>	<i>(Approximate) Frailty models</i>
----------------------	-------------------------------------

Description

When included in a [coxph](#) or [survreg](#), fits by penalised likelihood a random effects (frailty) model. `frailty` is generic, with methods for t, Gaussian and Gamma distributions.

Usage

```
frailty(x, distribution="gamma", ...)
frailty.gamma(x, sparse = (nclass > 5), theta, df, eps = 1e-05, method = c("em","aic", "df")
frailty.gaussian(x, sparse = (nclass > 5), theta, df, method = c("reml","aic", "df", "fixed")
frailty.t(x, sparse = (nclass > 5), theta, df, eps = 1e-05, tdf = 5, method = c("aic", "df",
```

Arguments

<code>x</code>	group indicator
<code>distribution</code>	frailty distribution
<code>...</code>	Arguments for specific distribution, including
<code>sparse</code>	Use sparse Newton-Raphson algorithm
<code>df</code>	Approximate degrees of freedom
<code>theta</code>	Penalty
<code>eps</code>	Accuracy of <code>df</code>
<code>method</code>	maximisation algorithm
<code>tdf</code>	df of t-distribution
<code>caic</code>	Use corrected AIC rather than AIC

Details

The penalised likelihood method is equivalent to maximum (partial) likelihood for the gamma frailty but not for the others.

The sparse algorithm uses the diagonal of the information matrix for the random effects, which saves a lot of space.

The frailty distributions are really the log-t and lognormal: t and Gaussian are random effects on the scale of the linear predictor.

Value

An object of class `coxph.penalty` containing a factor with attributes specifying the control functions.

See Also

[coxph,survreg,ridge,pspline](#)

Examples

```
data(kidney)
kfit <- coxph(Surv(time, status)~ age + sex + disease + frailty(id), kidney)
kfit0 <- coxph(Surv(time, status)~ age + sex + disease, kidney)
kfitm1 <- coxph(Surv(time,status) ~ age + sex + disease +
               frailty(id, dist='gauss'), kidney)
coxph(Surv(time, status) ~ age + sex + frailty(id, dist='gauss', method='aic',caic=T), kidney)
# uncorrected aic
coxph(Surv(time, status) ~ age + sex + frailty(id, method='aic', caic=F), kidney)

data(rats)
rfit2a <- survreg(Surv(time, status) ~ rx +
                  frailty.gaussian(litter, df=13, sparse=F), rats )
rfit2b <- survreg(Surv(time, status) ~ rx +
                  frailty.gaussian(litter, df=13, sparse=T), rats )

rfit2a
rfit2b
```

heart

Stanford Heart Transplant data

Description

Survival of patients on the waiting list for the Stanford heart transplant program.

Usage

```
data(heart)
data(jasa)
data(jasa1)
```

Format

jasa: original data

birth.dt:	birth date
accept.dt:	acceptance into program
tx.date:	transplant date
fu.date:	end of followup
fustat:	dead or alive
surgery:	prior bypass surgery
age:	age (in days)
futime:	followup time
wait.time:	time before transplant
transplant:	transplant indicator
mismatch:	mismatch score
hla.a2:	particular type of mismatch
mscore:	another mismatch score
reject:	rejection occurred

jasa1, heart: processed data

start, stop, event:	Entry and exit time and status for this interval of time
age:	age-48 years
year:	year of acceptance (in years after 1 Nov 1967)
surgery:	prior bypass surgery 1=yes
transplant:	received transplant 1=yes
id:	patient id

Source

Crowley and Hu, JASA

See Also

[stanford2](#)

is.ratetable	<i>Verify that an Object is of Class 'ratetable'</i>
--------------	--

Description

The function verifies not only the `class` attribute, but the structure of the object.

Usage

```
is.ratetable(x, verbose=FALSE)
```

Arguments

<code>x</code>	the object to be verified.
<code>verbose</code>	if <code>TRUE</code> and the object is not a <code>ratetable</code> , then return a character string describing the way(s) in which <code>x</code> fails to be a proper <code>ratetable</code> object.

Details

Rate tables are used by the `pyears` and `survexp` functions, and normally contain death rates for some population, categorized by age, sex, or other variables. They have a fairly rigid structure, and the `verbose` option can help in creating a new rate table.

Value

returns `TRUE` if `x` is a `ratetable`, and `FALSE` or a description if it is not.

See Also

[pyears](#), [survexp](#)

Examples

```
library(ratetables)
is.ratetable(survexp.us) ##Yes
data(cancer)
is.ratetable(cancer) ##No
```

kidney	<i>Kidney data from survival5</i>
--------	-----------------------------------

Description

Data on the recurrence times to infection, at the point of insertion of the catheter, for kidney patients using portable dialysis equipment. Catheters may be removed for reasons other than infection, in which case the observation is censored. Each patient has exactly 2 observations.

Format

```

patient:  id
time:     time
status:   event status
age:      in years
sex:      1=male, 2=female
disease:  disease type (0=GN, 1=AN, 2=PKD, 3=Other)
frail:    frailty estimate from original paper

```

Note

The original analysis had incorrect handling of ties and so is not exactly reproduced by survival.

Source

McGilchrist and Aisbett, Biometrics 47, 461-66, 1991

Examples

```

data(kidney)
kfit <- coxph(Surv(time, status)~ age + sex + disease + frailty(id), kidney)
kfit0 <- coxph(Surv(time, status)~ age + sex + disease, kidney)
kfitm1 <- coxph(Surv(time,status) ~ age + sex + disease +
               frailty(id, dist='gauss'), kidney)

```

lines.survfit

Add Lines or Points to a Survival Plot

Description

Often used to add the expected survival curve(s) to a Kaplan-Meier plot generated with plot.survfit.

Usage

```

lines(x, type="s", mark=3, col=1, lty=1,
      lwd=1, mark.time=TRUE, xscale=1, firstx=0, firsty=1, xmax, fun, conf.int=FALSE, ...)
points(x, ...)

```

Arguments

x	a survival object, generated from the <code>survfit</code> or <code>survexp</code> functions.
type	the line type, as described in <code>lines</code> . The default is a step function for <code>survfit</code> objects, and a connected line for <code>survexp</code> objects.
mark	vectors giving the mark symbol, color, line type and line width for the added curves.

<code>mark.time</code>	controls the labeling of the curves. If FALSE , no labeling is done. If TRUE , then curves are marked at each censoring time. If <code>mark.time</code> is a numeric vector, then curves are marked at the specified time points.
<code>xscale</code>	a number used to divide the x values. If time was originally in days, a value of 365.24 would give a plotted scale in years.
<code>firstx,firsty</code>	the starting point for the survival curves. If either of these is set to NA or <blank> the plot will start at the first time point of the curve.
<code>col,lty,lwd,...</code>	passed to lines
<code>xmax</code>	the maximum horizontal plot coordinate. This shortens the curve before plotting it, so unlike using the <code>xlim</code> graphical parameter, warning messages about out of bounds points are not generated.
<code>fun</code>	an arbitrary function defining a transformation of the survival curve. For example <code>fun=log</code> is an alternative way to draw a log-survival curve (but with the axis labeled with $\log(S)$ values). Four often used transformations can be specified with a character argument instead: "log" is the same as using the <code>log=T</code> option, "event" plots cumulative events ($f(y) = 1 - y$), "cumhaz" plots the cumulative hazard function ($f(y) = -\log(y)$) and "cloglog" creates a complementary log-log survival plot ($f(y) = \log(-\log(y))$) along with log scale for the x-axis).
<code>conf.int</code>	if TRUE , confidence bands for the curves are also plotted. If set to "only" , then only the CI bands are plotted, and the curve itself is left off. This can be useful for fine control over the colors or line types of a plot.

Value

a list with components **x** and **y**, containing the coordinates of the last point on each of the curves (but not of the confidence limits). This may be useful for labeling.

Side Effects

one or more curves are added to the current plot.

See Also

[lines](#), [par](#), [plot.survfit](#), [survfit](#), [survexp](#).

Examples

```
data(pbc)
fit <- survfit(Surv(time, status) ~ sex, pbc, subset=1:312)
plot(fit, mark.time=F, xscale=365.24,
      xlab='Years', ylab='Survival')
lines(fit[1], lwd=2, xscale=365.24)    #darken the first curve and add marks

# add expected survival curves for the two groups,
# based on the US expected
tdata <- data.frame(age=pbcs$age*365.24, sex=pbcs$sex +1,
                    year= rep(mdy.date(1,1,1976), nrow(pbc)))
tdata<-tdata[1:312,] ## only the randomised people, with no missing data
data(ratetables)
efit <- survexp(~ sex+ratetable(sex=sex,age=age,year=year), data=tdata, ratetable=survexp.us, times=(
temp <- lines(efit, lty=2, xscale=365.24, lwd=2:1)
```

```
text(temp, c("Male", "Female"), adj= -.1) #labels just past the ends
title(main="Primary Biliary Cirrhosis, Observed and Expected")
```

lung

Mayo Clinic Lung Cancer Data

Description

Survival in patients with lung cancer at Mayo Clinic. Performance scores rate how well the patient can perform usual daily activities.

Usage

```
data(lung)
data(cancer)
```

Format

```
inst:      Institution code
time:      Survival time in days
status:    censoring status 1=censored, 2=dead
age:       Age in years
sex:       Male=1 Female=2
ph.ecog:   ECOG performance score (0=good 5=dead)
ph.karno:  Karnofsky performance score (bad=0-good=100) rated by physician
pat.karno: Karnofsky performance score rated by patient
meal.cal:  Calories consumed at meals
wt.loss:   Weight loss in last six months
```

Source

Terry Therneau

mdy.date

Convert to Julian Dates

Description

Given a month, day, and year, returns the number of days since January 1, 1960.

Usage

```
mdy.date(month, day, year, nineteen = TRUE, fillday = FALSE,
          fillmonth = FALSE)
```


Arguments

<code>month</code>	vector of months.
<code>day</code>	vector of days.
<code>year</code>	vector of years.
<code>nineteen</code>	if <code>TRUE</code> , year values between 0 and 99 are assumed to be in the 20th century A.D.; otherwise, if <code>FALSE</code> , they are assumed to be in the 1st century A.D.
<code>fillday</code>	if <code>TRUE</code> , then missing days are replaced with 15.
<code>fillmonth</code>	if <code>TRUE</code> , then a missing month causes the month and day to be set to 7/1.

Details

The date functions are particularly useful in computing time spans, such as number of days on test, and similar functions can be found in other statistical packages. The baseline date of Jan 1, 1960 is, of course, completely arbitrary (it is the same one used by SAS).

The `fillday` and `fillmonth` options are perhaps useful only to the author and a very few others: we sometimes deal with patients whose birth date was in the 1800's, and only the month or even only the year is known. When the interval is greater than 80 years, a filler seems defensible.

Value

a vector of Julian dates.

References

Press, W. H., Teukolsky, S. A., Vetterling, W. T., and Flannery, B. P. (1992). *Numerical Recipes: The Art of Scientific Computing (Second Edition)*. Cambridge University Press.

See Also

[date.mmddyy](#), [date.ddmmyy](#), [date.mmddyyyy](#)

Examples

```
mdy.date(3, 10, 53)
xzt <- 1:10
xzy <- as.date(xzt)
test <- data.frame(x = xzt, date = xzy)
summary(test)
```

ovarian

Ovarian cancer survival data

Description

Survival in a randomised trial comparing two treatments for ovarian cancer (? from an Eastern Cooperative Oncology Group study)

Usage

```
data(ovarian)
```

Format

```
futime: survival or censoring time
fustat: censoring status
age:    in years
resid.ds: residual disease present (1=no,2=yes)
rx:     treatment group
ecog.ps: ECOG performance status (1 is better, see reference)
```

Source

Terry Therneau

References

for ECOG performance status: http://ecog.dfci.harvard.edu/~ecogdba/general/perf_stat.html

pbc	<i>Mayo Clinic Primary Biliary Cirrhosis Data</i>
-----	---

Description

Followup of 312 randomised and 108 unrandomised patients with primary biliary cirrhosis, a rare autoimmune liver disease, at Mayo Clinic.

Usage

```
data(pbc)
```

Format

```
age:    in years
alb:    serum albumin
alkphos: alkaline phosphatase
ascites: presence of ascites
bili:   serum bilirunbin
chol:   serum cholesterol
edema:  presence of edema
edtrt:  0 no edema, 0.5 untreated or successfully treated
        1 unsuccessfully treated edema
hepmeg: enlarged liver
time:   survival time
platelet: platelet count
protime: standardised blood clotting time
```

sex:	1=male
sgot:	liver enzyme (now called AST)
spiders:	blood vessel malformations in the skin
stage:	histologic stage of disease (needs biopsy)
status:	censoring status
trt:	1/2/-9 for control, treatment, not randomised
trig:	triglycerides
copper:	urine copper

Source

Fleming TR, Harrington DP. "Counting Processes and Survival Analysis" Wiley:New York

plot.cox.zph	<i>Graphical Test of Proportional Hazards</i>
--------------	---

Description

Displays a graph of the scaled Schoenfeld residuals, along with a smooth curve.

Usage

```
plot.cox.zph(x, resid=TRUE, se=TRUE, df=4, nsmo=40, var, ...)
```

Arguments

x	result of the <code>cox.zph</code> function.
resid	a logical value, if <code>TRUE</code> the residuals are included on the plot, as well as the smooth fit.
se	a logical value, if <code>TRUE</code> , confidence bands at two standard errors will be added.
df	the degrees of freedom for the fitted natural spline, <code>df=2</code> leads to a linear fit.
nsmo	number of points used to plot the fitted spline.
var	the set of variables for which plots are desired. By default, plots are produced in turn for each variable of a model. Selection of a single variable allows other features to be added to the plot, e.g., a horizontal line at zero or a main title. This has been superseded by a subscripting method; see the example below.
...	additional arguments passed to the <code>plot</code> function.

Side Effects

a plot is produced on the current graphics device.

See Also

[cox.zph](#), [coxph](#)

Examples

```
data(veteran)
vfit <- coxph(Surv(time,status) ~ trt + factor(celltype) +
              karno + age, data=veteran, x=T)
temp <- cox.zph(vfit)
plot(temp, var=5)      # Look at Karnofsky score, old way of doing plot
plot(temp[5])         # New way with subscripting
abline(0, 0, lty=3)
# Add the linear fit as well
abline(lm(temp$y[,5] ~ temp$x)$coefficients, lty=4, col=3)
title(main="VA Lung Study")
```

plot.survfit

Plot Method for 'survfit'

Description

A plot of survival curves is produced, one curve for each strata. The `log=T` option does extra work to avoid `log(0)`, and to try to create a pleasing result. If there are zeros, they are plotted by default at 0.8 times the smallest non-zero value on the curve(s).

If `legend.text` is supplied a legend is created.

Usage

```
plot.survfit(x, conf.int=<<see below>>, mark.time=TRUE,
             mark=3, col=1, lty=1, lwd=1, cex=1, log=FALSE, xscale=1, yscale=1,
             firstx=0, firsty=1, xmax, ymin=0, fun,
             xlab="", ylab="", xaxs="S", bty=NULL,
             legend.text=NULL, legend.pos=0, legend.bty="n", main=NULL, ...)
```

Arguments

<code>x</code>	an object of class <code>survfit</code> , usually returned by the <code>survfit</code> function.
<code>conf.int</code>	determines whether confidence intervals will be plotted. The default is to do so if there is only 1 curve, i.e., no strata.
<code>mark.time</code>	controls the labeling of the curves. If set to <code>FALSE</code> , no labeling is done. If <code>TRUE</code> , then curves are marked at each censoring time which is not also a death time. If <code>mark.time</code> is a numeric vector, then curves are marked at the specified time points.
<code>mark</code>	vector of mark parameters, which will be used to label the curves. The <code>lines</code> help file contains examples of the possible marks. The vector is reused cyclically if it is shorter than the number of curves.
<code>col</code>	a vector of integers specifying colors for each curve. The default value is 1.
<code>lty</code>	a vector of integers specifying line types for each curve. The default value is 1.

<code>lwd</code>	a vector of numeric values for line widths. The default value is 1.
<code>cex</code>	a numeric value specifying the size of the marks. Not a vector; all marks have the same size.
<code>log</code>	a logical value, if TRUE the y axis will be on a log scale. Alternately, one of the standard character strings "x", "y", or "xy" can be given to specific logarithmic horizontal and/or vertical axes.
<code>xscale</code>	scale the x-axis values before plotting. If time were in days, then a value of 365.25 will give labels in years instead of the original days.
<code>yscale</code>	will be used to multiply the labels on the y axis. A value of 100, for instance, would be used to give a percent scale. Only the labels are changed, not the actual plot coordinates, so that adding a curve with <code>"lines(surv.exp(...{ }))"</code> , say, will perform as it did without the <code>yscale</code> argument.
<code>firstx,firsty</code>	the starting point for the survival curves. If either of these is set to NA the plot will start at the first time point of the curve.
<code>xmax</code>	the maximum horizontal plot coordinate. This can be used to shrink the range of a plot. It shortens the curve before plotting it, so that unlike using the <code>xlim</code> graphical parameter, warning messages about out of bounds points are not generated.
<code>ymin</code>	lower boundary for y values. Survival curves are most often drawn in the range of 0-1, even if none of the curves approach zero. The parameter is ignored if the <code>fun</code> argument is present, or if it has been set to NA.
<code>fun</code>	an arbitrary function defining a transformation of the survival curve. For example <code>fun=log</code> is an alternative way to draw a log-survival curve (but with the axis labeled with $\log(S)$ values), and <code>fun=sqrt</code> would generate a curve on square root scale. Four often used transformations can be specified with a character argument instead: "log" is the same as using the <code>log=T</code> option, "event" plots cumulative events ($f(y) = 1-y$), "cumhaz" plots the cumulative hazard function ($f(y) = -\log(y)$), and "cloglog" creates a complimentary log-log survival plot ($f(y) = \log(-\log(y))$) along with log scale for the x-axis).
<code>xlab</code>	label given to the x-axis.
<code>ylab</code>	label given to the y-axis.
<code>axs</code>	either "S" for a survival curve or a standard x axis style as listed in <code>par</code> . Survival curves are usually displayed with the curve touching the y-axis, but not touching the bounding box of the plot on the other 3 sides. Type "S" accomplishes this by manipulating the plot range and then using the "i" style internally.
<code>bty</code>	see par
<code>legend.pos</code>	position for the legend: a vector of length 2, or 0 to put the legend in the lower left, 1 to put it in the upper right. These may not work well with transformed curves.
<code>legend.bty</code>	Box type, see legend
<code>legend.text</code>	Text for legend, see legend
<code>main</code>	Plot title
<code>...</code>	other graphical parameters

Value

a list with components `x` and `y`, containing the coordinates of the last point on each of the curves (but not the confidence limits). This may be useful for labeling.

BUGS

Survival curve objects created from a `coxph` model does not include the censoring times. Therefore, specifying `mark.time=T` does not work. If you want to mark censoring times on the curve(s) resulting from a `coxph` fit, provide a vector of times as the `mark.time` argument in the call to `plot.survfit` or `lines.survfit`.

See Also

[par](#), [survfit](#), [lines.survfit](#) [legend](#)

Examples

```
data(aml)
leukemia.surv <- survfit(Surv(time, status) ~ x, data = aml)
plot(leukemia.surv, lty = 2:3)
legend(100, .9, c("Maintenance", "No Maintenance"), lty = 2:3)
title("Kaplan-Meier Curves\nfor AML Maintenance Study")

lsurv2 <- survfit(Surv(time, status) ~ x, aml, type='fleming')
plot(lsurv2, lty=2:3, fun="cumhaz",
      xlab="Months", ylab="Cumulative Hazard")

plot(leukemia.surv, lty=1:2, legend.pos=0, col=c("Red", "Blue"), legend.text=c("Maintenance",
"No Maintenance"))
if (interactive()){
  cat("Click on the graphics device to place a legend\n")
  plot(leukemia.surv, lty=2:3, legend.pos=locator(1), legend.text=c("Maintenance",
"No Maintenance"))
}
```

predict.coxph

Predictions for Cox model

Description

Compute fitted values and regression terms for a model fitted by [coxph](#)

Usage

```
predict.coxph(object, newdata, type=c("lp", "risk", "expected", "terms"), se.fit=FALSE, term
```

Arguments

<code>object</code>	A <code>coxph</code> object
<code>newdata</code>	Optional new data to predict at
<code>type</code>	Type of prediction wanted
<code>se.fit</code>	Return standard errors as well?

<code>terms</code>	If <code>type="terms"</code> , which terms to return.
<code>collapse</code>	identifier for which rows correspond to different individuals
<code>safe</code>	Use a more intensive ‘safe’ prediction method
<code>...</code>	other unused arguments

Value

A vector or matrix of fitted values. If `se.fit=TRUE` a list whose first component is the fitted values and second is the standard errors.

See Also

[predict,coxph,termplot](#)

Examples

```
data(aml)
fit<-coxph(Surv(time,status)~x,data=aml)
predict(fit,type="lp")
predict(fit,type="risk")
predict(fit,type="expected")
predict(fit,type="terms")
predict(fit,type="lp",se.fit=TRUE)
predict(fit,type="risk",se.fit=TRUE)
predict(fit,type="expected",se.fit=TRUE)
predict(fit,type="terms",se.fit=TRUE)
```

<code>predict.survreg</code>	<i>Predicted Values for a ‘survreg’ Object</i>
------------------------------	--

Description

Predicted values for a `survreg` object

Usage

```
predict(object, newdata,
  type=c("response", "link", "linear", "response", "terms", "quantile",
        "uquantile"),
  se.fit=F, terms=labels.lm(object), p=c(0.1, 0.9),...)
```

Arguments

<code>object</code>	result of a model fit using the <code>survreg</code> function.
<code>newdata</code>	data for prediction. If absent, predictions are for the subjects used in the original fit.
<code>type</code>	the type of predicted value. This can be on the original scale of the data (<code>response</code>), the linear predictor (<code>"linear"</code> , with <code>"lp"</code> as an allowed abbreviation), a predicted quantile on the original scale of the data (<code>"quantile"</code>), a quantile on the linear predictor scale (<code>"uquantile"</code>), or the matrix of terms for the linear predictor (<code>"terms"</code>). At this time <code>"link"</code> and linear predictor (<code>"lp"</code>) are identical.

<code>se.fit</code>	if TRUE, include the standard errors of the prediction in the result.
<code>terms</code>	subset of terms. The default for residual type " <code>terms</code> " is a matrix with one column for every term (excluding the intercept) in the model.
<code>p</code>	vector of percentiles. This is used only for quantile predictions.
<code>...</code>	other arguments

Value

a vector or matrix of predicted values.

References

Escobar and Meeker (1992). Assessing influence in regression analysis with censored data. *Biometrics*, 48, 507-528.

See Also

[survreg](#), [residuals.survreg](#)

Examples

```
# Draw figure 1 from Escobar and Meeker
data(stanford2)
fit <- survreg(Surv(time,status) ~ age + age^2, data=stanford2,
               dist='lognormal')
plot(stanford2$age, stanford2$time, xlab='Age', ylab='Days',
     xlim=c(0,65), ylim=c(.01, 10^6), log='y')
pred <- predict(fit, newdata=list(age=1:65), type='quantile',
               p=c(.1, .5, .9))
matlines(1:65, pred, lty=c(2,1,2), col=1)
```

`print.survfit`

Print a Short Summary of a Survival Curve

Description

Print number of observations, number of events, the restricted mean survival and its standard error, and the median survival with confidence limits for the median.

Usage

```
print(x, scale=1,digits = max(options())$digits - 4,
      3), print.n=getOption("survfit.print.n"), show.rmean=getOption("survfit.print.mean"),...)
```

Arguments

<code>x</code>	the result of a call to the <code>survfit</code> function.
<code>print.n</code>	What to use for number of subjects (see below)
<code>digits</code>	Number of digits to print
<code>scale</code>	a numeric value to rescale the survival time, e.g., if the input data to <code>survfit</code> were in days, <code>scale=365</code> would scale the printout to years.
<code>show.rmean</code>	Show the restricted mean survival?
<code>...</code>	other unused arguments

Details

The restricted mean (**rmean**) and its standard error **se(rmean)** are based on a truncated estimator. If the last observation(s) is not a death, then the survival curve estimate does not go to zero and the mean survival time cannot be estimated. Instead, the quantity reported is the mean of survival restricted to the time before the last censoring. When the last censoring time is not random this quantity is occasionally of interest.

Any randomness in the last censoring time is not taken into account in computing the standard error of the restricted mean. The restricted mean is shown mainly for compatibility with S, and may not be shown by default in future versions.

The median and its confidence interval are defined by drawing a horizontal line at 0.5 on the plot of the survival curve and its confidence bands. The intersection of the line with the lower CI band defines the lower limit for the median's interval, and similarly for the upper band. If any of the intersections is not a point, then we use the smallest point of intersection, e.g., if the survival curve were exactly equal to 0.5 over an interval.

The "number of observations" is not well-defined for counting process data. Previous versions of this code used the number at risk at the first time point. This is misleading if many individuals enter late or change strata. The original S code for the current version uses the number of records, which is misleading when the counting process data actually represent a fixed cohort with time-dependent covariates.

Four possibilities are provided, controlled by **print.n** or by **options(survfit.print.n)**: "none" prints NA, "records" prints the number of records, "start" prints the number at the first time point and "max" prints the maximum number at risk. The initial default is "start".

Value

x, with the invisible flag set.

Side Effects

The number of observations (see Details), the number of events, the restricted mean survival (**rmean**) and its standard error, and the median survival with its confidence interval are printed. If there are multiple curves, there is one line of output for each.

See Also

[summary.survfit](#), [survfit.object](#), [survfit](#)

Examples

```
##effect of print.n and show.rmean
data(heart)
a<-coxph(Surv(start,stop,event)~age+strata(transplant),data=heart)
b<-survfit(a)
print(b,print.n="none")
print(b,print.n="records")
print(b,print.n="start")
print(b,print.n="max")
print(b,show.rmean=FALSE)
```

pspline

*Penalised smoothing splines***Description**

Specifies a penalised spline basis for the predictor. This is done by fitting a comparatively small set of splines and penalising the integrated second derivative. Results are similar to smoothing splines with a knot at each data point but computationally simpler.

Usage

```
pspline(x, df=4, theta, nterm=2.5 * df, degree=3, eps=0.1, method, ...)
```

Arguments

x	predictor
df	approximate degrees of freedom. df=0 means use AIC
theta	roughness penalty
nterm	number of splines in the basis
degree	degree of splines
eps	accuracy for df
method	Method for automatic choice of theta
...	I don't know what this does

Value

Object of class `coxph.penalty` containing the spline basis with attributes specifying control functions.

See Also

[coxph](#), [survreg](#), [ridge](#), [frailty](#)

Examples

```
data(cancer)
lfit6 <- survreg(Surv(time, status)~pspline(age, df=2), cancer)
plot(cancer$age, predict(lfit6), xlab='Age', ylab="Spline prediction")
title("Cancer Data")
fit0 <- coxph(Surv(time, status) ~ ph.ecog + age, cancer)
fit1 <- coxph(Surv(time, status) ~ ph.ecog + pspline(age,3), cancer)
fit3 <- coxph(Surv(time, status) ~ ph.ecog + pspline(age,8), cancer)
fit0
fit1
fit3
```

pyears	<i>Person Years</i>
---------------	---------------------

Description

This function computes the person-years of follow-up time contributed by a cohort of subjects, stratified into subgroups. It also computes the number of subjects who contribute to each cell of the output table, and optionally the number of events and/or expected number of events in each cell.

Usage

```
pyears(formula, data, weights, subset, na.action, ratetable=survexp.us,
scale=365.25, expect=c('event', 'pyears'), model=FALSE, x=FALSE, y=FALSE)
```

Arguments

formula	a formula object. The response variable will be a vector of follow-up times for each subject, or a Surv object containing the follow-up time and an event indicator. The predictors consist of optional grouping variables separated by + operators (exactly as in survfit), time-dependent grouping variables such as age (specified with tcut), and optionally a ratetable() term. This latter matches each subject to his/her expected cohort.
data	a data frame in which to interpret the variables named in the formula , or in the subset and the weights argument.
weights	case weights.
subset	expression saying that only a subset of the rows of the data should be used in the fit.
na.action	a missing-data filter function, applied to the model.frame, after any subset argument has been used. Default is options()\$na.action .
ratetable	a table of event rates, such as survexp.uswhite .
scale	a scaling for the results. As most rate tables are in units/day, the default value of 365.25 causes the output to be reported in years.
expect	should the output table include the expected number of events, or the expected number of person-years of observation. This is only valid with a rate table.
model,x,y	If any of these is true, then the model frame, the model matrix, and/or the vector of response times will be returned as components of the final result.

Details

Because **pyears** may have several time variables, it is necessary that all of them be in the same units. For instance, in the call `py <- pyears(futime ~ rx + ratetable(age=age, sex=sex, year=entry.dt))` with a **ratetable** whose natural unit is days, it is important that **futime**, **age** and **entry.dt** all be in days. Given the wide range of possible inputs, it is difficult for the routine to do sanity checks of this aspect.

A special function **tcut** is needed to specify time-dependent cutpoints. For instance, assume that age is in years, and that the desired final arrays have as one of their margins the age

groups 0-2, 2-10, 10-25, and 25+. A subject who enters the study at age 4 and remains under observation for 10 years will contribute follow-up time to both the 2-10 and 10-25 subsets. If `cut(age, c(0,2,10,25,100))` were used in the formula, the subject would be classified according to his starting age only. The `tcut` function has the same arguments as `cut`, but produces a different output object which allows the `pyears` function to correctly track the subject.

The results of `pyears` are normally used as input to further calculations. The example below is from a study of hip fracture rates from 1930 - 1990 in Rochester, Minnesota. Survival post hip fracture has increased over that time, but so has the survival of elderly subjects in the population at large. A model of relative survival helps to clarify what has happened: Poisson regression is used, but replacing exposure time with expected exposure (for an age and sex matched control). Death rates change with age, of course, so the result is carved into 1 year increments of time. Males and females were done separately.

Value

a list with components

<code>pyears</code>	an array containing the person-years of exposure. (Or other units, depending on the rate table and the scale).
<code>n</code>	an array containing the number of subjects who contribute time to each cell of the <code>pyears</code> array.
<code>event</code>	an array containing the observed number of events. This will be present only if the response variable is a <code>Surv</code> object.
<code>expected</code>	an array containing the expected number of events (or person years). This will be present only if there was a <code>ratetable</code> term.
<code>offtable</code>	the number of person-years of exposure in the cohort that was not part of any cell in the <code>pyears</code> array. This is often useful as an error check; if there is a mismatch of units between two variables, nearly all the person years may be off table.
<code>summary</code>	a summary of the rate-table matching. This is also useful as an error check.
<code>call</code>	an image of the call to the function.
<code>na.action</code>	the <code>na.action</code> attribute contributed by an <code>na.action</code> routine, if any.

See Also

[ratetable](#), [survexp](#), [Surv](#)

Examples

```
data(ratetables)
#
# Simple case: a single male subject, born 6/6/36 and entered on study 6/6/55.
#

temp1 <- mdy.date(6,6,36)
temp2 <- mdy.date(6,6,55)
# Now compare the results from person-years
#
temp.age <- tcut(temp2-temp1, floor(c(-1, (18:31 * 365.24)))),
               labels=c('0-18', paste(18:30, 19:31, sep='-')))
```

```
temp.yr <- tcut(temp2, mdy.date(1,1,1954:1965), labels=1954:1964)
temp.time <- 3700 #total days of fu
py1 <- pyears(temp.time ~ temp.age + temp.yr, scale=1) #output in days
data(ratetables)
survexp.uswhite<-survexp.usr[,,"white",]
py2 <- pyears(temp.time ~ temp.age + temp.yr
              + ratetable(age=temp2-temp1, year=temp2, sex=1),
              scale=1, ratetable=survexp.uswhite ) #output in days
```

ratetable

Ratetable reference in formula

Description

This function matches variable names in data to those in a ratetable for [survexp](#)

Usage

```
ratetable(...)
```

Arguments

... tags matching dimensions of the ratetable and variables in the data frame
(see example)

Value

A data frame

See Also

[survexp](#), [survexp.us](#), [is.ratetable](#)

Examples

```
data(pbc)
fit <- survfit(Surv(time, status) ~ sex, pbc, subset=1:312)
if (require(date)){
  tdata <- data.frame(agedays=pbc$age*365.24, sex=pbc$sex +1,
                    year= rep(mdy.date(1,1,1976), nrow(pbc)))
  tdata<-tdata[1:312,] ## only the randomised people, with no missing data
  data(ratetables)
  efit <- survexp(~ sex+ratetable(sex=sex,age=agedays,year=year), data=tdata, ratetable=survexp.us, time)
}
```

<code>rats</code>	<i>Rat data from survival5</i>
-------------------	--------------------------------

Description

48 rats were injected with a carcinogen, and then randomized to either drug or placebo. The number of tumors ranges from 0 to 13; all rats were censored at 6 months after randomization.

Usage

```
data(rats)
```

Format

<code>rat:</code>	<code>id</code>
<code>rx:</code>	<code>treatment,(1=drug, 0=control)</code>
<code>observation:</code>	<code>within rat</code>
<code>start:</code>	<code>entry time</code>
<code>stop:</code>	<code>exit time</code>
<code>status:</code>	<code>event status</code>

Source

Gail, Sautner and Brown, Biometrics 36, 255-66, 1980

<code>residuals.coxph</code>	<i>Calculate Residuals for a ‘coxph’ Fit</i>
------------------------------	--

Description

Calculates martingale, deviance, score or Schoenfeld residuals for a Cox proportional hazards model.

Usage

```
residuals(object,
  type=c("martingale", "deviance", "score", "schoenfeld",
        "dfbeta", "dfbetas", "scaledsch"),
  collapse=FALSE, weighted=FALSE, ...)
residuals(object,
  type=c("martingale", "deviance"),
  collapse=FALSE, weighted=FALSE, ...)
```

Arguments

object	an object inheriting from class <code>coxph</code> , representing a fitted Cox regression model. Typically this is the output from the <code>coxph</code> function.
type	character string indicating the type of residual desired. Possible values are "martingale", "deviance", "score", "schoenfeld", "dfbeta", "dfbetas", and "scaledsch". Only enough of the string to determine a unique match is required.
collapse	vector indicating which rows to collapse (sum) over. In time-dependent models more than one row data can pertain to a single individual. If there were 4 individuals represented by 3, 1, 2 and 4 rows of data respectively, then <code>collapse=c(1,1,1, 2, 3,3, 4,4,4,4)</code> could be used to obtain per subject rather than per observation residuals.
weighted	if <code>TRUE</code> and the model was fit with case weights, then the weighted residuals are returned.
...	other unused arguments

Value

For martingale and deviance residuals, the returned object is a vector with one element for each subject (without `collapse`). For score residuals it is a matrix with one row per subject and one column per variable. The row order will match the input data for the original fit. For Schoenfeld residuals, the returned object is a matrix with one row for each event and one column per variable. The rows are ordered by time within strata, and an attribute `strata` is attached that contains the number of observations in each strata. The scaled Schoenfeld residuals are used in the `cox.zph` function.

The score residuals are each individual's contribution to the score vector. Two transformations of this are often more useful: `dfbeta` is the approximate change in the coefficient vector if that observation were dropped, and `dfbetas` is the approximate change in the coefficients, scaled by the standard error for the coefficients.

NOTE

For deviance residuals, the status variable may need to be reconstructed. For score and Schoenfeld residuals, the X matrix will need to be reconstructed.

References

T. Therneau, P. Grambsch, and T. Fleming. "Martingale based residuals for survival models", *Biometrika*, March 1990.

See Also

[coxph](#)

Examples

```
data(heart)
fit <- coxph(Surv(start, stop, event) ~ (age + surgery)* transplant,
             data=heart)
mresid <- resid(fit, collapse=heart$id)
```

residuals.survreg	<i>Compute Residuals for 'survreg' Objects</i>
-------------------	--

Description

This is a method for the function `residuals` for objects inheriting from class `survreg`.

Usage

```
residuals.survreg(object, type, rsigma=T, collapse=F, weighted=F,...)
```

Arguments

<code>object</code>	an object inheriting from class <code>survreg</code> .
<code>type</code>	type of residuals, with choices of "response", "deviance", "dfbeta", "dfbetas", "working", "ldcase", "lsresp", "ldshape", and "matrix". See the LaTeX documentation (<code>survival/doc/survival.ps.gz</code>) for more detail.
<code>rsigma</code>	include the scale parameters in the variance matrix, when doing computations. (I can think of no good reason not to).
<code>collapse</code>	optional vector of subject groups. If given, this must be of the same length as the residuals, and causes the result to be per group residuals.
<code>weighted</code>	give weighted residuals? Normally residuals are unweighted.
<code>...</code>	other unused arguments

Value

A vector or matrix of residuals is returned. Response residuals are on the scale of the original data, working residuals are on the scale of the linear predictor, and deviance residuals are on log-likelihood scale. The dfbeta residuals are a matrix, where the *i*th row gives the approximate change in the coefficients due to the addition of subject *i*. The dfbetas matrix contains the dfbeta residuals, with each column scaled by the standard deviation of that coefficient.

The matrix type produces a matrix based on derivatives of the log-likelihood function. Let L be the log-likelihood, p be the linear predictor $X\beta$, and s be $\log(\sigma)$. Then the 6 columns of the matrix are L , dL/dp , $\partial^2 L/\partial p^2$, dL/ds , $\partial^2 L/\partial s^2$ and $\partial^2 L/\partial p \partial s$. Diagnostics based on these quantities are discussed in an article by Escobar and Meeker. The main ones are the likelihood displacement residuals for perturbation of a case weight (`ldcase`), the response value (`ldresp`), and the `shape`.

References

Escobar, L. A. and Meeker, W. Q. (1992). Assessing influence in regression analysis with censored data. *Biometrics* **48**, 507-528.

See Also

`predict.survreg`

Examples

```
data(aml)
fit <- survreg(Surv(time,status) ~x, aml)
rr <- residuals(fit, type='matrix')
```

ridge

Ridge regression

Description

When used in a [coxph](#) or [survreg](#) model formula, specifies a ridge regression term. The likelihood is penalised by `theta/2` time the sum of squared coefficients. If `scale=T` the penalty is calculated for coefficients based on rescaling the predictors to have unit variance. If `df` is specified then `theta` is chosen based on an approximate degrees of freedom.

Usage

```
ridge(..., theta, df=nvar/2, eps=0.1, scale=T)
```

Arguments

<code>...</code>	predictors to be ridged
<code>theta</code>	penalty is <code>theta/2</code> time sum of squared coefficients
<code>df</code>	Approximate degrees of freedom
<code>eps</code>	Accuracy required for <code>df</code>
<code>scale</code>	Scale variables before applying penalty?

Value

An object of class `coxph.penalty` containing the data and control functions.

References

Gray (1992) "Flexible methods of analysing survival data using splines, with applications to breast cancer prognosis" JASA 87:942-951

See Also

[coxph](#), [survreg](#), [pspline](#), [frailty](#)

Examples

```
data(ovarian)
fit1 <- coxph(Surv(futime, fustat) ~ rx + ridge(age, ecog.ps, theta=1),
             ovarian)

fit1
data(cancer)
lfit0 <- survreg(Surv(time, status) ~1, cancer)
lfit1 <- survreg(Surv(time, status) ~ age + ridge(ph.ecog, theta=5), cancer)
lfit2 <- survreg(Surv(time, status) ~ sex + ridge(age, ph.ecog, theta=1), cancer)
lfit3 <- survreg(Surv(time, status) ~ sex + age + ph.ecog, cancer)
```

```
lfit0
lfit1
lfit2
lfit3
```

stanford2
More Stanford Heart Transplant data

Description

This contains the Stanford Heart Transplant data in a different format. The main data set is in [heart](#)

Usage

```
data(stanford2)
```

Format

id:	ID number
time:	survival or censoring time
status:	censoring status
age:	in years
t5:	T5 mismatch score

Source

Escobar & Meeker, Biometrics 1992 p519

See Also

[predict.survreg,heart](#)

strata
Identify Strata Variables

Description

This is a special function used in the context of the Cox survival model. It identifies stratification variables when they appear on the right hand side of a formula.

Usage

```
strata(..., na.group=F, shortlabel=F)
```

Arguments

... any number of variables. All must be the same length.
 na.group a logical variable, if TRUE, then missing values are treated as a distinct level of each variable.
 shortlabel if TRUE omit variable names from resulting factor labels

Details

The result is identical to the `interaction` function, but for the labeling of the factors (`strata` is more verbose).

Value

a new factor, whose levels are all possible combinations of the factors supplied as arguments.

See Also

[coxph](#), [interaction](#)

Examples

```
a<-factor(rep(1:3,4))
b<-factor(rep(1:4,3))
levels(strata(a))
levels(strata(a,b,shortlabel=TRUE))
data(ovarian)
coxph(Surv(futime, fustat) ~ age + strata(rx), data=ovarian)
```

summary.survfit

Summary of a Survival Curve

Description

Returns a list containing the survival curve, confidence limits for the curve, and other information.

Usage

```
summary.survfit(object, times, censored=F, scale=1, ...)
```

Arguments

object output from a call to `survfit`.
 times vector of times; the returned matrix will contain 1 row for each time. This must be in increasing order and missing values are not allowed. If `censored=T`, the default `times` vector contains all the unique times in `fit`, otherwise the default `times` vector uses only the event (death) times.
 censored logical flag: should the censoring times be included in the output? This is ignored if the `times` argument is present.
 scale rescale the survival time, e.g., if the input data to `survfit` were in days, `scale=365.25` would scale the output to years.
 extend Estimate survival beyond the last observed time?
 ... other unused arguments

Value

a list with the following components

See Also

[survfit](#), [print.summary.survfit](#).

Examples

```
data(ovarian)
summary( survfit( Surv(futime, fustat),data=ovarian))
```

survdif

Test Survival Curve Differences

Description

Tests if there is a difference between two or more survival curves using the G^p family of tests, or for a single curve against a known alternative.

Usage

```
survdif(formula, data, subset, na.action, rho=0)
```

Arguments

formula	a formula expression as for other survival models, of the form Surv(time, status) ~ predictors . For a one-sample test, the predictors must consist of a single offset(sp) term, where sp is a vector giving the survival probability of each subject. For a k-sample test, each unique combination of predictors defines a subgroup. A strata term may be used to produce a stratified test. To cause missing values in the predictors to be treated as a separate group, rather than being omitted, use the strata function with its na.group=T argument.
data	an optional data frame in which to interpret the variables occurring in the formula.
subset	expression indicating which subset of the rows of data should be used in the fit. This can be a logical vector (which is replicated to have length equal to the number of observations), a numeric vector indicating which observation numbers are to be included (or excluded if negative), or a character vector of row names to be included. All observations are included by default.
na.action	a missing-data filter function. This is applied to the model.frame after any subset argument has been used. Default is options()\$na.action .
rho	a scalar parameter that controls the type of test.

Value

a list with components:

n	the number of subjects in each group.
obs	the weighted observed number of events in each group. If there are strata, this will be a matrix with one column per stratum.
exp	the weighted expected number of events in each group. If there are strata, this will be a matrix with one column per stratum.
chisq	the chisquare statistic for a test of equality.
var	the variance matrix of the test.
strata	optionally, the number of subjects contained in each stratum.

METHOD

This function implements the G-rho family of Harrington and Fleming (1982), with weights on each death of $S(t)^{\rho}$, where $S(t)$ is the Kaplan-Meier estimate of survival. With **rho** = 0 this is the log-rank or Mantel-Haenszel test, and with **rho** = 1 it is equivalent to the Peto & Peto modification of the Gehan-Wilcoxon test.

If the right hand side of the formula consists only of an offset term, then a one sample test is done. To cause missing values in the predictors to be treated as a separate group, rather than being omitted, use the **factor** function with its **exclude** argument.

References

Harrington, D. P. and Fleming, T. R. (1982). A class of rank test procedures for censored survival data. *Biometrika* **69**, 553-566.

Examples

```
## Two-sample test
data(ovarian)
survdiff(Surv(futime, fustat) ~ rx, data=ovarian)
rm(ovarian)
## Stratified 7-sample test
data(lung)
survdiff(Surv(time, status) ~ pat.karno + strata(inst), data=lung)
rm(lung)

data(heart)
data(ratetables)
## Expected survival for heart transplant patients based on
## US mortality tables
expect <- survexp(futime ~ ratetable(age=(accept.dt - birth.dt),
sex=1, year=accept.dt, race="white"), jasa, cohort=F,
ratetable=survexp.usr)
## actual survival is much worse (no surprise)
print(survdiff(Surv(jasa$futime, jasa$fustat) ~ offset(expect)))
rm(jasa, jasa1, heart, survexp.az, survexp.azr, survexp.fl, survexp.flr, survexp.mn, survexp.mnwhite, survexp
```

survexp	<i>Compute Expected Survival</i>
---------	----------------------------------

Description

Returns either the expected survival of a cohort of subjects, or the individual expected survival for each subject.

Usage

```
survexp(formula, data, weights, subset, na.action, times, cohort=T,
        conditional=F, ratetable=survexp.us, scale=1, npoints,
        se.fit=<<see below>>, model=F, x=F, y=F)
```

Arguments

formula	formula object. The response variable is a vector of follow-up times and is optional. The predictors consist of optional grouping variables separated by the + operator (as in survfit), along with a ratetable term. The ratetable term matches each subject to his/her expected cohort.
data	data frame in which to interpret the variables named in the formula , subset and weights arguments.
weights	case weights.
subset	expression indicating a subset of the rows of data to be used in the fit.
na.action	function to filter missing data. This is applied to the model frame after subset has been applied. Default is options()\$na.action . A possible value for na.action is na.omit , which deletes observations that contain one or more missing values.
times	vector of follow-up times at which the resulting survival curve is evaluated. If absent, the result will be reported for each unique value of the vector of follow-up times supplied in formula .
cohort	logical value: if FALSE , each subject is treated as a subgroup of size 1. The default is TRUE .
conditional	logical value: if TRUE , the follow-up times supplied in formula are death times and conditional expected survival is computed. If FALSE , the follow-up times are potential censoring times. If follow-up times are missing in formula , this argument is ignored.
ratetable	a table of event rates, such as survexp.uswhite , or a fitted Cox model.
scale	numeric value to scale the results. If ratetable is in units/day, scale = 365.25 causes the output to be reported in years.
npoints	number of points at which to calculate intermediate results, evenly spaced over the range of the follow-up times. The usual (exact) calculation is done at each unique follow-up time. For very large data sets specifying npoints can reduce the amount of memory and computation required. For a prediction from a Cox model npoints is ignored.
se.fit	compute the standard error of the predicted survival. The default is to compute this whenever the routine can, which at this time is only for the Ederer method and a Cox model as the rate table.

`model,x,y` flags to control what is returned. If any of these is true, then the model frame, the model matrix, and/or the vector of response times will be returned as components of the final result, with the same names as the flag arguments.

Details

Individual expected survival is usually used in models or testing, to 'correct' for the age and sex composition of a group of subjects. For instance, assume that birth date, entry date into the study, sex and actual survival time are all known for a group of subjects. The `survexp.uswhite` population tables contain expected death rates based on calendar year, sex and age. Then `haz <- -log(survexp(death.time ratetable(sex=sex, year=entry.dt, age=(birth.dt-entry.dt)), cohort=F))` gives for each subject the total hazard experienced up to their observed death time or censoring time. This probability can be used as a rescaled time value in models: `glm(status ~ 1 + offset(log(haz)), family=poisson)` `glm(status ~ x + offset(log(haz)), family=poisson)` In the first model, a test for `intercept=0` is the one sample log-rank test of whether the observed group of subjects has equivalent survival to the baseline population. The second model tests for an effect of variable `x` after adjustment for age and sex.

Cohort survival is used to produce an overall survival curve. This is then added to the Kaplan-Meier plot of the study group for visual comparison between these subjects and the population at large. There are three common methods of computing cohort survival. In the "exact method" of Ederer the cohort is not censored; this corresponds to having no response variable in the formula. Hakulinen recommends censoring the cohort at the anticipated censoring time of each patient, and Verheul recommends censoring the cohort at the actual observation time of each patient. The last of these is the conditional method. These are obtained by using the respective time values as the follow-up time or response in the formula.

Value

if `cohort=T` an object of class `survexp`, otherwise a vector of per-subject expected survival values. The former contains the number of subjects at risk and the expected survival for the cohort at each requested time.

References

G. Berry. The analysis of mortality by the subject-years method. *Biometrics* 1983, 39:173-84. F Ederer, L Axtell, and S Cutler. The relative survival rate: a statistical methodology. *Natl Cancer Inst Monogr* 1961, 6:101-21. T. Hakulinen. Cancer survival corrected for heterogeneity in patient withdrawal. *Biometrics* 1982, 38:933. H. Verheul, E. Dekker, P. Bossuyt, A. Moulijn, and A. Dunning. Background mortality in clinical survival studies. *Lancet* 1993, 341:872-5.

See Also

[survfit](#), [survexp.us](#), [survexp.fit](#), [pyears](#), [date](#)

Examples

```
## comparing data to population
data(ratetables)
data(cancer)
## compare survival to US population
```

```

cancer$year<-rep(as.date("1/1/1980"),nrow(cancer))
efit <- survexp( ~ ratetable(sex=sex, year=year, age=age*365), times=(1:4)*365,data=cancer)
plot(survfit(Surv(time, status) ~1,data=cancer))
lines(efit)
## compare data to Cox model
## Mayo PBC data
data(pbc)
## fit to randomised patients
m<-coxph(Surv(time,status)~edtrt+log(bili)+log(protime)+age+platelet,data=dbc,
         subset=trt>0)
##compare Kaplan-Meier to fitted model for 2 edema groups in
##unrandomised patients
plot(survfit(Surv(time,status)~edtrt,data=dbc,subset=trt==9))
lines(survexp(~edtrt+ratetable(edtrt=edtrt,bili=bili,platelet=platelet,age=age,
                             protime=protime),data=dbc,subset=trt==9,ratetable=m,cohort=TRUE),col="purple")

```

survexp.fit

Work Function to Compute Expected Survival

Description

Compute expected survival. This function is not to be called by the user.

Usage

```
survexp.fit(x, y, times, death, ratetable)
```

Arguments

x	a matrix. The first column contains the group, an integer value that divides the subjects into subsets. Remaining columns must match the dimensions of the ratetable , in the correct order.
y	the follow up time for each subject.
times	the vector of times at which a result will be computed.
death	death indicator
ratetable	a rate table, such as survexp.uswhite .

Details

For cohort survival it must be the potential censoring time for each subject, ignoring death.

For an exact estimate **times** should be a superset of **y**, so that each subject at risk is at risk for the entire sub-interval of time. For a large data set, however, this can use an inordinate amount of storage and/or compute time. If the **times** spacing is more coarse than this, an actuarial approximation is used which should, however, be extremely accurate as long as all of the returned values are $> .99$.

For a subgroup of size 1 and **times** $> y$, the conditional method reduces to $\exp(-h)$ where h is the expected cumulative hazard for the subject over his/her observation time. This is used to compute individual expected survival.

Value

A list containing the number of subjects and the expected survival(s) at each time point. If there are multiple groups, these will be matrices with one column per group.

WARNING

Most users will call the higher level routine **survexp**. Consequently, this function has very few error checks on its input arguments.

See Also

[survexp](#), [survexp.us](#)

ratetables	<i>Census Data Sets for the Expected Survival and Person Years Functions</i>
-------------------	--

Description

Census data sets for the expected survival and person years functions

Usage

```
data(ratetables)
survexp.uswhite<-survexp.usr[,,"white",]
```

Details

us total United States population, by age and sex, 1960 to 1980.

uswhite United States white population, by age and sex, 1950 to 1980. This is no longer included, but can be extracted from **survexp.usr** as shown above.

usr United States population, by age, sex and race, 1960 to 1980. Race is white, nonwhite, or black. For 1960 and 1970 the black population values were not reported separately, so the nonwhite values were used.

mn total Minnesota population, by age and sex, 1970 and 1980.

mnwhite Minnesota white population, by age and sex, 1960 to 1980.

fl total Florida population, by age and sex, 1970 and 1980.

flr Florida population, by age, sex and race, 1970-1980. Race is white, nonwhite, or black. For 1970 the black population values were not reported separately, so the nonwhite values were used.

az total Arizona population, by age and sex, 1970 and 1980.

azr Arizona population, by age, sex and race, 1970-1980. Race is white versus nonwhite. For 1970 the nonwhite population values were not reported separately. In order to make the rate table be a matrix, the 1980 values were repeated. (White and non-white values are quite different).

Each of these tables contains the daily hazard rate for a matched subject from the population, defined as $-\log(1-q)/365.24$ where q is the 1 year probability of death as reported in the original tables. For age 25 in 1970, for instance, $p = 1-q$ is the probability that a subject who becomes 25 years of age in 1970 will achieve his/her 26th birthday. The tables

are recast in terms of hazard per day entirely for computational convenience. (The fraction .24 in the denominator is based on 24 leap years per century.)

Each table is stored as an array, with additional attributes, and can be subset and manipulated as standard S arrays. Interpolation between calendar years is done "on the fly" by the survexp routine.

Some of the deficiencies, e.g. 1970 Arizona non-white, are a result of local conditions. The data probably exists, but we don't have a copy it in the library.

The tables have been augmented to contain extrapolated values for 1990 and 2000. The details can be found in Mayo Clinic Biostatistics technical report 63 at <http://www.mayo.edu/hsr/techrpt.html>

survfit

Compute a Survival Curve for Censored Data

Description

Computes an estimate of a survival curve for censored data using either the Kaplan-Meier or the Fleming-Harrington method or computes the predicted survivor function for a Cox proportional hazards model.

Usage

```
survfit(formula, data, weights, subset, na.action,
        newdata, individual=F, conf.int=.95, se.fit=T,
        type=c("kaplan-meier", "fleming-harrington", "fh2"),
        error=c("greenwood", "tsiatis"),
        conf.type=c("log", "log-log", "plain", "none"),
        conf.lower=c("usual", "peto", "modified"))
basehaz(fit, centered=TRUE)
```

Arguments

formula	A formula object or a <code>coxph</code> object. If a formula object is supplied it must have a <code>Surv</code> object as the response on the left of the <code>~</code> operator and, if desired, terms separated by <code>+</code> operators on the right. One of the terms may be a <code>strata</code> object. For a single survival curve the " <code>~ 1</code> " part of the formula is not required.
data	a data frame in which to interpret the variables named in the formula, or in the <code>subset</code> and the <code>weights</code> argument.
weights	The weights must be nonnegative and it is strongly recommended that they be strictly positive, since zero weights are ambiguous, compared to use of the <code>subset</code> argument.
subset	expression saying that only a subset of the rows of the data should be used in the fit.
na.action	a missing-data filter function, applied to the model frame, after any <code>subset</code> argument has been used. Default is <code>options()\$na.action</code> .

<code>newdata</code>	a data frame with the same variable names as those that appear in the <code>coxph</code> formula. Only applicable when <code>formula</code> is a <code>coxph</code> object. The curve(s) produced will be representative of a cohort who's covariates correspond to the values in <code>newdata</code> . Default is the mean of the covariates used in the <code>coxph</code> fit.
<code>individual</code>	a logical value indicating whether the data frame represents different time epochs for only one individual (T), or whether multiple rows indicate multiple individuals (F, the default). If the former only one curve will be produced; if the latter there will be one curve per row in <code>newdata</code> .
<code>conf.int</code>	the level for a two-sided confidence interval on the survival curve(s). Default is 0.95.
<code>se.fit</code>	a logical value indicating whether standard errors should be computed. Default is TRUE.
<code>type</code>	a character string specifying the type of survival curve. Possible values are "kaplan-meier", "fleming-harrington" or "fh2" if a formula is given and "aalen" or "kaplan-meier" if the first argument is a <code>coxph</code> object, (only the first two characters are necessary). The default is "aalen" when a <code>coxph</code> object is given, and it is "kaplan-meier" otherwise.
<code>error</code>	either the string "greenwood" for the Greenwood formula or "tsiatis" for the Tsiatis formula, (only the first character is necessary). The default is "tsiatis" when a <code>coxph</code> object is given, and it is "greenwood" otherwise.
<code>conf.type</code>	One of "none", "plain", "log" (the default), or "log-log". Only enough of the string to uniquely identify it is necessary. The first option causes confidence intervals not to be generated. The second causes the standard intervals <code>curve +/- k * se(curve)</code> , where k is determined from <code>conf.int</code> . The log option calculates intervals based on the cumulative hazard or log(survival). The last option bases intervals on the log hazard or log(-log(survival)). These last will never extend past 0 or 1.
<code>conf.lower</code>	controls modified lower limits to the curve, the upper limit remains unchanged. The modified lower limit is based on an 'effective n' argument. The confidence bands will agree with the usual calculation at each death time, but unlike the usual bands the confidence interval becomes wider at each censored observation. The extra width is obtained by multiplying the usual variance by a factor m/n , where n is the number currently at risk and m is the number at risk at the last death time. (The bands thus agree with the un-modified bands at each death time.) This is especially useful for survival curves with a long flat tail. The Peto lower limit is based on the same 'effective n' argument as the modified limit, but also replaces the usual Greenwood variance term with a simple approximation. It is known to be conservative.
<code>fit</code>	a <code>coxph</code> object
<code>centered</code>	Compute the baseline hazard at the covariate mean rather than at zero?

Details

Actually, the estimates used are the Kalbfleisch-Prentice (Kalbfleisch and Prentice, 1980, p.86) and the Tsiatis/Link/Breslow, which reduce to the Kaplan-Meier and Fleming-Harrington estimates, respectively, when the weights are unity. When curves are fit for a Cox model, subject weights of `exp(sum(coef*(x-center)))` are used, ignoring any value for `weights`

input by the user. There is also an extra term in the variance of the curve, due to the variance of the coefficients and hence variance in the computed weights.

The Greenwood formula for the variance is a sum of terms $d/(n*(n-m))$, where d is the number of deaths at a given time point, n is the sum of `weights` for all individuals still at risk at that time, and m is the sum of `weights` for the deaths at that time. The justification is based on a binomial argument when weights are all equal to one; extension to the weighted case is ad hoc. Tsiatis (1981) proposes a sum of terms $d/(n*n)$, based on a counting process argument which includes the weighted case.

The two variants of the F-H estimate have to do with how ties are handled. If there were 3 deaths out of 10 at risk, then the first would increment the hazard by $3/10$ and the second by $1/10 + 1/9 + 1/8$. For curves created after a Cox model these correspond to the Breslow and Efron estimates, respectively, and the proper choice is made automatically. The `fh2` method will give results closer to the Kaplan-Meier.

Based on the work of Link (1984), the log transform is expected to produce the most accurate confidence intervals. If there is heavy censoring, then based on the work of Dorey and Korn (1987) the modified estimate will give a more reliable confidence band for the tails of the curve.

Value

a `survfit` object; see the help on `survfit.object` for details. Methods defined for `survfit` objects are provided for `print`, `plot`, `lines`, and `points`.

For `basehaz`, a dataframe with the baseline hazard, times, and strata.

References

- Dorey, F. J. and Korn, E. L. (1987). Effective sample sizes for confidence intervals for survival probabilities. *Statistics in Medicine* 6, 679-87.
- Fleming, T. H. and Harrington, D.P. (1984). Nonparametric estimation of the survival distribution in censored data. *Comm. in Statistics* 13, 2469-86.
- Kalbfleisch, J. D. and Prentice, R. L. (1980). *The Statistical Analysis of Failure Time Data*. Wiley, New York.
- Link, C. L. (1984). Confidence intervals for the survival function using Cox's proportional hazards model with covariates. *Biometrics* 40, 601-610.
- Tsiatis, A. (1981). A large sample study of the estimate for the integrated hazard function in Cox's regression model for survival data. *Annals of Statistics* 9, 93-108.

See Also

[print.survfit](#), [plot.survfit](#), [lines.survfit](#), [summary.survfit](#), [coxph](#), [Surv](#), [strata](#).

Examples

```
#fit a Kaplan-Meier and plot it
data(aml)
fit <- survfit(Surv(time, status) ~ x, data=aml)
plot(fit)

# plot only 1 of the 2 curves from above
plot(fit[2])

#fit a cox proportional hazards model and plot the
```

```
#predicted survival curve
data(ovarian)
fit <- coxph( Surv(futime,fustat)~resid.ds+rx+ecog.ps,data=ovarian)
plot( survfit( fit))
```

survfit.object

Survival Curve Object

Description

This class of objects is returned by the `survfit` class of functions to represent a fitted survival curve.

Objects of this class have methods for the functions `print`, `summary`, `plot`, `points` and `lines`. The `print.survfit` method does more computation than is typical for a `print` method and is documented on a separate page.

COMPONENTS

The following components must be included in a legitimate `survfit` object.

time the time points at which the curve has a step.

n.risk the number of subjects at risk at t.

n.event the number of events that occur at time t.

surv the estimate of survival at time t+0. This may be a vector or a matrix.

strata if there are multiple curves, this component gives the number of elements of the `time` etc. vectors corresponding to the first curve, the second curve, and so on. The names of the elements are labels for the curves.

std.err the standard error of the cumulative hazard or $-\log(\text{survival})$.

upper upper confidence limit for the survival curve.

lower lower confidence limit for the survival curve.

conf.type the approximation used to compute the confidence limits.

conf.int the level of the confidence limits, e.g. 90 or 95%.

na.action the returned value from the `na.action` function, if any. It will be used in the printout of the curve, e.g., the number of observations deleted due to missing values.

call the call that produced the object.

SUBSCRIPTS

Survfit objects that contain multiple survival curves can be subscripted. This is most often used to plot a subset of the curves. Usually a single subscript will be used. In one particular case – survival curves for multiple covariate values, from a Cox model that includes a `strata` statement – there is a matrix of curves and 2 subscripts may be used. (In this case `summary.survfit` will also print the data as a matrix).

See Also

`survfit`, `plot.survfit`, `summary.survfit`, `print.survfit`.

survival-internal	<i>Internal survival functions</i>
-------------------	------------------------------------

Description

Internal survival functions

Usage

```
survreg.fit(x, y, weights, offset, init, controlvals, dist, scale = 0,
            nstrat = 1, strata, parms = NULL)
survpenal.fit(x, y, weights, offset, init, controlvals, dist, scale = 0,
              nstrat = 1, strata, pcols, pattr, assign, parms = NULL)
survdifffit(y, x, strat, rho = 0)
is.category(x)
termobj[i]
match.ratetable(R, ratetable)
as.matrix.ratetable(x)
is.na.ratetable2(x)
is.na.coxph.penalty(x)
model.newframe(object, newdata, safe = F, response = F, ...)
as.data.frame.difftime(x, row.names = NULL, optional = FALSE)
coxpenal.df(hmat, hinv, fdiag, assign.list, ptype, nvar, pen1,
            pen2, sparse)
coxpenal.fit(x, y, strata, offset, init, control, weights, method,
             rownames, pcols, pattr, assign)
coxph.fit(x, y, strata, offset, init, control, weights, method,
          rownames)
coxph.wtest(var, b, toler.chol = 1e-09)
agreg.fit(x, y, strata, offset, init, control, weights, method,
          rownames)
agexact.fit(x, y, strata, offset, init, control, weights, method,
            rownames)
```

Details

These are not to be called by the user

survobrien	<i>O'Brien's Test for Association of a Single Variable with Survival</i>
------------	--

Description

Peter O'Brien's test for association of a single variable with survival This test is proposed in Biometrics, June 1978.

Usage

```
survobrien(formula, data)
```

Arguments

formula	a valid formula for a cox model, without time dependent covariates.
data	a data frame.

Value

a new data frame. The original time and status variables are removed, and have been replaced with **start**, **stop**, and **event**. If a predictor variable is a factor or is protected with **I()**, it is retained as is. Other predictor variables have been replaced with time-dependent logit scores.

Because of the time dependent variables, the new data frame will have many more rows than the original data, approximately $\#rows * \#deaths / 2$.

METHOD

A time-dependent cox model can now be fit to the new data. The univariate statistic, as originally proposed, is equivalent to single variable score tests from the time-dependent model. This equivalence is the rationale for using the time dependent model as a multivariate extension of the original paper.

In O'Brien's method, the x variables are re-ranked at each death time. A simpler method, proposed by Prentice, ranks the data only once at the start. The results are usually similar.

References

O'Brien, Peter, "A Nonparametric Test for Association with Censored Data", *Biometrics* 34: 243-250, 1978.

See Also

[survdiff](#)

Examples

```
data(ovarian)
xx <- survobrien(Surv(futime, fustat) ~ age + factor(rx) + I(ecog.ps),
                 data=ovarian)
coxph(Surv(start, stop, event) ~ age, data=xx)
coxph(Surv(start, stop, event) ~ age + rx + ecog.ps, data=xx)
```

survreg

Regression for a Parametric Survival Model

Description

Regression for a parametric survival model. These are all time-transformed location models, with the most useful case being the accelerated failure models that use a log transformation.

Usage

```
survreg(formula=formula(data), data=parent.frame(), weights,
subset, na.action, dist="weibull", init=NULL, scale=0,
control=survreg.control(), parms=NULL, model=FALSE, x=FALSE,
y=TRUE, ...)
```

Arguments

<code>formula</code>	a formula expression as for other regression models. See the documentation for <code>lm</code> and <code>formula</code> for details.
<code>data</code>	optional data frame in which to interpret the variables occurring in the formula.
<code>weights</code>	Optional observation weights
<code>subset</code>	subset of the observations to be used in the fit.
<code>na.action</code>	function to be used to handle any NAs in the data.
<code>dist</code>	assumed distribution for y variable. If the argument is a character string, then it is assumed to name an element from survreg.distributions . These include "weibull", "exponential", "gaussian", "logistic", "lognormal" and "loglogistic". Otherwise, it is assumed to be a user defined list conforming to the format described in survreg.distributions .
<code>parms</code>	a list of fixed parameters. For the t-distribution for instance this is the degrees of freedom; most of the distributions have no parameters.
<code>init</code>	optional vector of initial values for the parameters.
<code>scale</code>	optional fixed value for the scale. If set to ≤ 0 then the scale is estimated.
<code>control</code>	a list of control values, in the format produced by <code>survreg.control</code> .
<code>model</code>	if TRUE, the model frame is returned.
<code>x</code>	if TRUE, then the X matrix is returned.
<code>y</code>	if TRUE, then the y vector (or survival times) is returned.
<code>...</code>	other arguments which will be passed to <code>survreg.control</code> .

Value

an object of class `survreg` is returned.

Compatibility note

This routine underwent significant changes from survival4 to survival5. The [survreg.old](#) function gives a backwards-compatible interface. In S-PLUS the new function is called `survReg` and the old one `survreg`.

See Also

[survreg.object](#), [survreg.distributions](#), [pspline](#), [frailty](#), [ridge](#), [survreg.old](#)

Examples

```
data(ovarian)
## These are all the same
survreg(Surv(futime, fustat) ~ ecog.ps + rx, ovarian, dist='weibull',scale=1)
survreg(Surv(futime, fustat) ~ ecog.ps + rx, ovarian,
dist="exponential")
survreg.old(Surv(futime, fustat) ~ ecog.ps + rx, ovarian, dist='extreme',fixed=list(scale=1),link="log
```

survreg.control	<i>Package options for survreg and coxph</i>
-----------------	--

Description

These functions check and package the fitting options for [survreg](#) and [coxph](#)

Usage

```
survreg.control(maxiter=30, rel.tolerance=1e-05, failure=1,  
toler.chol=1e-10, iter.max, debug=0, outer.max=10)  
coxph.control (eps = 1e-04, toler.chol = .Machine$double.eps^0.75,  
iter.max = 10, toler.inf = sqrt(eps), outer.max = 10)
```

Arguments

<code>maxiter</code>	maximum number of iterations
<code>rel.tolerance</code>	relative tolerance to declare convergence
<code>failure</code>	value of status variable indicating failure
<code>toler.chol</code>	Tolerance to declare Cholesky decomposition singular
<code>iter.max</code>	same as <code>maxiter</code>
<code>debug</code>	print debugging information
<code>outer.max</code>	maximum number of outer iterations for choosing penalty parameters
<code>eps</code>	Tolerance to declare convergence for Cox model
<code>toler.inf</code>	An undocumented parameter

Value

A list with the same elements as the input

See Also

[survreg](#), [coxph](#)

Examples

survreg.distributions

Parametric Survival Distributions

Description

List of distributions for accelerated failure models. These are location-scale families for some transformation of time. The entry describes the cdf F and density f of a canonical member of the family.

Usage

survreg.distributions

Format

There are three basic formats; only the first two are used in the built-in distributions

name:	name of distribution
variance:	Variance
init(x,weights,...):	Function returning an initial
mean and	variance
deviance(y,scale,parms):	Function returning the deviance
density(x,parms):	Function returning F , $1 - F, f, f'/f, f''/f$
quantile(p,parms):	Quantile function
scale:	Optional fixed value for scale parameter

and for transformations of the time variable

name:	name of distribution
dist:	name of transformed distribution
trans:	transformation (eg log)
dtrans:	derivative of transformation
itrans:	inverse of transformation
scale:	Optional fixed value for scale parameter

For transformations of user-defined families use

name:	name of distribution
dist:	transformed distribution in first format
trans:	transformation (eg log)
dtrans:	derivative of transformation
itrans:	inverse of transformation
scale:	Optional fixed value for scale parameter

Details

There are four basic distributions: **extreme**, **gaussian**, **logistic** and **t**. The last three are parametrised in the same way as the distributions already present in R. The extreme value

cdf is

$$F = 1 - e^{-e^t}.$$

When the logarithm of survival time has one of the first three distributions we obtain respectively `lognormal`, `loglogistic` and `weibull`.

The other predefined distributions are defined in terms of these. The `exponential` and `rayleigh` distributions are Weibull distributions with fixed `scale` of 1 and 0.5 respectively, and `loggaussian` is a synonym for `lognormal`.

Parts of the built-in distributions are hardcoded in C, so the elements of `survreg.distributions` in the first format above must not be changed and new ones must not be added. The examples show how to specify user-defined distributions to `survreg`.

See Also

`survreg`, `pnorm`, `plogis`, `pt`

Examples

```
data(ovarian)
## not a good fit, but a useful example
survreg(Surv(futime,fustat)~ecog.ps+rx,data=ovarian,dist='extreme')
##
my.extreme<-survreg.distributions$extreme
my.extreme$name<-"Xtreme"
survreg(Surv(futime,fustat)~ecog.ps+rx,data=ovarian,dist=my.extreme)

## time transformation
survreg(Surv(futime,fustat)~ecog.ps+rx,data=ovarian,dist='weibull',scale=1)
my.weibull<-survreg.distributions$weibull
my.weibull$dist<-my.extreme
survreg(Surv(futime,fustat)~ecog.ps+rx,data=ovarian,dist=my.weibull,scale=1)

## change the transformation to work in years
## intercept changes by log(365), other coefficients stay the same
my.weibull$trans<-function(y) log(y/365)
my.weibull$itrans<-function(y) exp(365*y)
survreg(Surv(futime,fustat)~ecog.ps+rx,data=ovarian,dist=my.weibull,scale=1)
```

`survreg.object`

Parametric Survival Model Object

Description

This class of objects is returned by the `survreg` function to represent a fitted parametric survival model. Objects of this class have methods for the functions `print`, `summary`, `predict`, and `residuals`.

COMPONENTS

The following components must be included in a legitimate **survreg** object.

coefficients the coefficients of the **linear.predictors**, which multiply the columns of the model matrix. It does not include the estimate of error (sigma). The names of the coefficients are the names of the single-degree-of-freedom effects (the columns of the model matrix). If the model is over-determined there will be missing values in the coefficients corresponding to non-estimable coefficients.

icoef coefficients of the baseline model, which will contain the intercept and log(scale), or multiple scale factors for a stratified model.

var the variance-covariance matrix for the parameters, including the log(scale) parameter(s).

loglik a vector of length 2, containing the log-likelihood for the baseline and full models.

iter the number of iterations required

linear.predictors the linear predictor for each subject.

df the degrees of freedom for the final model. For a penalized model this will be a vector with one element per term.

scale the scale factor(s), with length equal to the number of strata.

idf degrees of freedom for the initial model.

means a vector of the column means of the coefficient matrix.

dist the distribution used in the fit.

The object will also have the following components found in other model results (some are optional): **linear predictors**, **weights**, **x**, **y**, **model**, **call**, **terms** and **formula**. See **lm**.

See Also

[survreg](#), [lm](#)

survreg.old

Old (survival4) Regression for a parametric survival model

Description

This routine is a backwards-compatible interface to the improved [survreg](#) function, which is better.

Usage

```
survreg.old(formula, data=sys.frame(sys.parent()), ...,
link=c("log", "identity"),
dist=c("extreme", "logistic", "gaussian", "exponential"),
fixed)
```

Arguments

formula	a formula expression as for other regression models. See the documentation for <code>lm</code> and formula for details.
data	optional data frame in which to interpret the variables occurring in the formula.
...	other arguments to survreg
link	transformation to be used on the y variable.
dist	assumed distribution for the transformed y variable.
fixed	a list of fixed parameters, most often just the scale. (When I implement the t-dist, it will include the degrees of freedom).

Value

an object of class `survreg` is returned, which inherits from class `glm`.

Examples

```
data(ovarian)
survreg.old(Surv(futime, fustat) ~ ecog.ps + rx, ovarian, dist='extreme',
            link='log', fixed=list(scale=1))  #Fit an exponential
```

tcut	<i>Factors for person-year calculations</i>
-------------	---

Description

Attaches categories for person-year calculations to a variable without losing the underlying continuous representation

Usage

```
tcut(x, breaks, labels, scale=1)
```

Arguments

x	numeric/date variable
breaks	breaks between categories, which are right-continuous
labels	labels for categories
scale	Multiply x and breaks by this.

Value

An object of class `tcut`

See Also

[cut](#), [pyears](#)

Examples

```
temp1 <- mdy.date(6,6,36)
temp2 <- mdy.date(6,6,55) # Now compare the results from person-years
#
temp.age <- tcut(temp2-temp1, floor(c(-1, (18:31 * 365.24)))),
               labels=c('0-18', paste(18:30, 19:31, sep='-')))
temp.yr <- tcut(temp2, mdy.date(1,1,1954:1965), labels=1954:1964)
temp.time <- 3700 #total days of fu
py1 <- pyyears(temp.time ~ temp.age + temp.yr, scale=1) #output in days
py1
```

untangle.specials	<i>Help Process the ‘specials’ Argument of the ‘terms’ Function.</i>
-------------------	--

Description

Given a `terms` structure and a desired special name, this returns an index appropriate for subscripting the `terms` structure and another appropriate for the data frame.

Usage

```
untangle.specials(tt, special, order=1)
```

Arguments

<code>tt</code>	a <code>terms</code> object.
<code>special</code>	the name of a special function, presumably used in the <code>terms</code> object.
<code>order</code>	the order of the desired terms. If set to 2, interactions with the special function will be included.
<code>vars</code>	a vector of variable names, as would be found in the data frame, of the specials.
<code>terms</code>	a numeric vector, suitable for subscripting the <code>terms</code> structure, that indexes the terms in the expanded model formula which involve the special.

Value

a list with two components:

Examples

```
formula<-Surv(tt,ss)~x+z*strata(id)
tms<-terms(formula,specials="strata")
## the specials attribute
attr(tms,"specials")
## main effects
untangle.specials(tms,"strata")
## and interactions
untangle.specials(tms,"strata",order=1:2)
```

veteran*Veterans' Administration Lung Cancer study*

Description

Randomised trial of two treatment regimens for lung cancer. This is a standard survival analysis data set

Usage

```
data(veteran)
```

Format

trt:	1=standard 2=test
celltype:	1=squamous, 2=smallcell, 3=adeno, 4=large
time:	survival time
status:	censoring status
karno:	Karnofsky performance score (100=good)
diagtime:	months from diagnosis to randomisation
age:	in years
prior:	prior therapy 0=no, 1=yes

Source

Kalbfleisch and Prentice "The Statistical Analysis of Failure Time Data"

Index

- *Topic **chron**
 - as.date, 3
 - date.ddmmyy, 14
 - date.mdy, 15
 - date.mmddyy, 15
 - date.mmddyyyy, 16
 - date.object, 17
 - mdy.date, 23
- *Topic **datasets**
 - aml, 3
 - bladder, 5
 - heart, 19
 - lung, 23
 - ovarian, 24
 - pbc, 25
 - ratetables, 48
 - rats, 37
 - stanford2, 41
 - veteran, 62
- *Topic **hplot**
 - plot.survfit, 27
- *Topic **internal**
 - survival-internal, 53
- *Topic **manip**
 - tcut, 60
- *Topic **models**
 - attrassign, 4
 - clogit, 6
- *Topic **regression**
 - survreg.object, 58
- *Topic **survival**
 - bladder, 5
 - clogit, 6
 - cluster, 7
 - colon, 7
 - cox.zph, 8
 - coxph, 9
 - coxph.detail, 12
 - coxph.object, 13
 - frailty, 17
 - heart, 19
 - is.ratetable, 20
 - kidney, 20
 - lines.survfit, 21
 - ovarian, 24
 - plot.cox.zph, 26
 - plot.survfit, 27
 - predict.coxph, 29
 - predict.survreg, 30
 - print.survfit, 31
 - pspline, 33
 - pyears, 34
 - ratetable, 36
 - ratetables, 48
 - rats, 37
 - residuals.coxph, 37
 - residuals.survreg, 39
 - ridge, 40
 - stanford2, 41
 - strata, 41
 - summary.survfit, 42
 - Surv, 1
 - survdiff, 43
 - survexp, 45
 - survexp.fit, 47
 - survfit, 49
 - survfit.object, 52
 - survival-internal, 53
 - survobrien, 53
 - survreg, 54
 - survreg.control, 56
 - survreg.distributions, 57
 - survreg.object, 58
 - survreg.old, 59
 - tcut, 60
 - untangle.specials, 61
- [.Surv (*Surv*), 1
- [.cox.zph (*cox.zph*), 8
- [.coxph.penalty (*coxph*), 9
- [.date (*date.object*), 17
- [.ratetable (*ratetable*), 36
- [.ratetable2 (*ratetable*), 36
- [.survfit (*survfit*), 49
- [.tcut (*tcut*), 60
- [.terms (*survival-internal*), 53
- [[.date (*date.object*), 17

- agexact.fit (*survival-internal*), 53
- agreg.fit (*survival-internal*), 53
- aml, 3
- anova.survreg (*survreg*), 54
- anova.survreglist (*survreg*), 54
- as.character.date (*date.object*), 17
- as.character.Surv (*Surv*), 1
- as.data.frame.date (*date.object*), 17
- as.data.frame.difftime (*survival-internal*), 53
- as.data.frame.Surv (*Surv*), 1
- as.date, 3, 17
- as.matrix.ratetable (*survival-internal*), 53
- as.vector.date (*date.object*), 17
- attrassign, 4
- basehaz (*survfit*), 49
- bladder, 5
- bladder2 (*bladder*), 5
- cancer (*lung*), 23
- ensorReg (*survreg*), 54
- clogit, 6
- cluster, 7, 11
- colon, 7
- cox.zph, 8, 14, 27
- coxpenal.df (*survival-internal*), 53
- coxpenal.fit (*survival-internal*), 53
- coxph, 2, 6, 7, 9, 9, 13, 14, 17, 18, 27, 29, 30, 33, 38, 40, 40, 42, 51, 56
- coxph.control, 10
- coxph.control (*survreg.control*), 56
- coxph.detail, 12, 14
- coxph.fit (*survival-internal*), 53
- coxph.object, 13
- coxph.wtest (*survival-internal*), 53
- cut, 60
- date, 46
- date.ddmmyy, 4, 14, 16, 17, 24
- date.mdy, 14, 15, 16, 17
- date.mmddyy, 4, 15, 24
- date.mmddyyyy, 16, 24
- date.object, 17
- format.Surv (*Surv*), 1
- frailty, 11, 11, 17, 33, 40, 55
- glm, 6
- heart, 19, 41
- interaction, 42
- is.category (*survival-internal*), 53
- is.date (*date.object*), 17
- is.na.coxph.penalty (*survival-internal*), 53
- is.na.date (*date.object*), 17
- is.na.ratetable (*ratetable*), 36
- is.na.ratetable2 (*survival-internal*), 53
- is.na.Surv (*Surv*), 1
- is.ratetable, 20, 36
- is.Surv (*Surv*), 1
- jasa (*heart*), 19
- jasa1 (*heart*), 19
- kidney, 20
- labels.survreg (*survreg*), 54
- legend, 28, 29
- leukemia (*aml*), 3
- lines, 22
- lines.survfit, 21, 29, 51
- lm, 59
- lung, 23
- match.ratetable (*survival-internal*), 53
- Math.date (*date.object*), 17
- Math.ratetable (*is.ratetable*), 20
- Math.Surv (*Surv*), 1
- mdy.date, 4, 14, 16, 17, 23
- model.frame.coxph (*coxph*), 9
- model.frame.survreg (*survreg*), 54
- model.matrix, 4
- model.newframe (*survival-internal*), 53
- Ops.date (*date.object*), 17
- Ops.ratetable (*is.ratetable*), 20
- Ops.Surv (*Surv*), 1
- ovarian, 24
- par, 22, 28, 29
- pbcr, 25
- plogis, 58
- plot.cox.zph, 26
- plot.date (*date.object*), 17
- plot.survfit, 22, 27, 51, 52
- pnorm, 58
- points.survfit (*lines.survfit*), 21
- predict, 30
- predict.coxph, 29
- predict.survreg, 30, 39, 41
- print.cox.zph (*cox.zph*), 8

print.coxph (*coxph.object*), 13
 print.coxph.null (*coxph*), 9
 print.coxph.penal (*coxph*), 9
 print.date (*date.object*), 17
 print.ratetable (*ratetable*), 36
 print.summary.survfit, 43
 print.summary.survfit
 (*summary.survfit*), 42
 print.summary.survreg (*survreg*), 54
 print.Surv (*Surv*), 1
 print.survdiff (*survdiff*), 43
 print.survexp (*survexp*), 45
 print.survfit, 31, 51, 52
 print.survreg (*survreg.object*), 58
 print.survreg.penal (*survreg*), 54
 pspline, 11, 11, 18, 33, 40, 55
 pt, 58
 pyyears, 20, 34, 46, 60

 ratetable, 35, 36
 ratetables, 48
 rats, 37
 residuals, 39
 residuals.coxph, 13, 14, 37
 residuals.survreg, 31, 39
 ridge, 11, 11, 18, 33, 40, 55

 stanford2, 19, 41
 strata, 6, 11, 41, 51
 summary.coxph (*coxph.object*), 13
 summary.coxph.penal (*coxph*), 9
 Summary.date (*date.object*), 17
 summary.date (*date.object*), 17
 summary.ratetable (*ratetable*), 36
 Summary.Surv (*Surv*), 1
 summary.survfit, 32, 42, 51, 52
 summary.survreg (*survreg.object*), 58
 Surv, 1, 7, 9, 11, 35, 51
 survdiff, 43, 54
 survdiff.fit (*survival-internal*), 53
 survexp, 20, 22, 35, 36, 45, 48
 survexp.az (*ratetables*), 48
 survexp.azr (*ratetables*), 48
 survexp.fit, 46, 47
 survexp.fl (*ratetables*), 48
 survexp.flr (*ratetables*), 48
 survexp.mn (*ratetables*), 48
 survexp.mnwhite (*ratetables*), 48
 survexp.us, 36, 46, 48
 survexp.us (*ratetables*), 48
 survexp.usr (*ratetables*), 48
 survexp.uswhite (*ratetables*), 48
 survexp.wnc (*ratetables*), 48

 survfit, 2, 11, 14, 22, 29, 32, 43, 46, 49, 52
 survfit.coxph.null (*survfit*), 49
 survfit.km (*survfit*), 49
 survfit.object, 32, 52
 survival-internal, 53
 survobrien, 53
 survpenal.fit (*survival-internal*), 53
 survReg (*survreg*), 54
 survreg, 2, 14, 17, 18, 31, 33, 40, 40, 54, 56, 58, 59, 59, 60
 survreg.control, 56
 survreg.distributions, 55, 57
 survreg.fit (*survival-internal*), 53
 survreg.object, 55, 58
 survreg.old, 55, 55, 59

 tcut, 60
 termplot, 30
 terms, 4

 untangle.specials, 61

 veteran, 62